Bring Your Own Git (BYOG) — Edge Delivery Services with AEM authoring

One codebase in a private GitHub Enterprise repository onboarded in Cloud Manager, many Edge Delivery sites (dev / QA / stage / prod), each pointing at its own AEM author environment.

Organization (aem.live org)	`gehealthcare-eds`
Canonical site / repo	`geh-brand`
Environment sites (examples)	`geh-brand-qa`, `geh-brand-stage`
Cloud Manager program id	`<program-id>`
Cloud Manager repository id	`<repository-id>`
AEM author (canonical)	`https://author-p<program-id>-e<env-id>.adobeaemcloud.com`
AEM author (QA example)	`https://author-p<program-id>-e<qa-env-id>.adobeaemcloud.com`

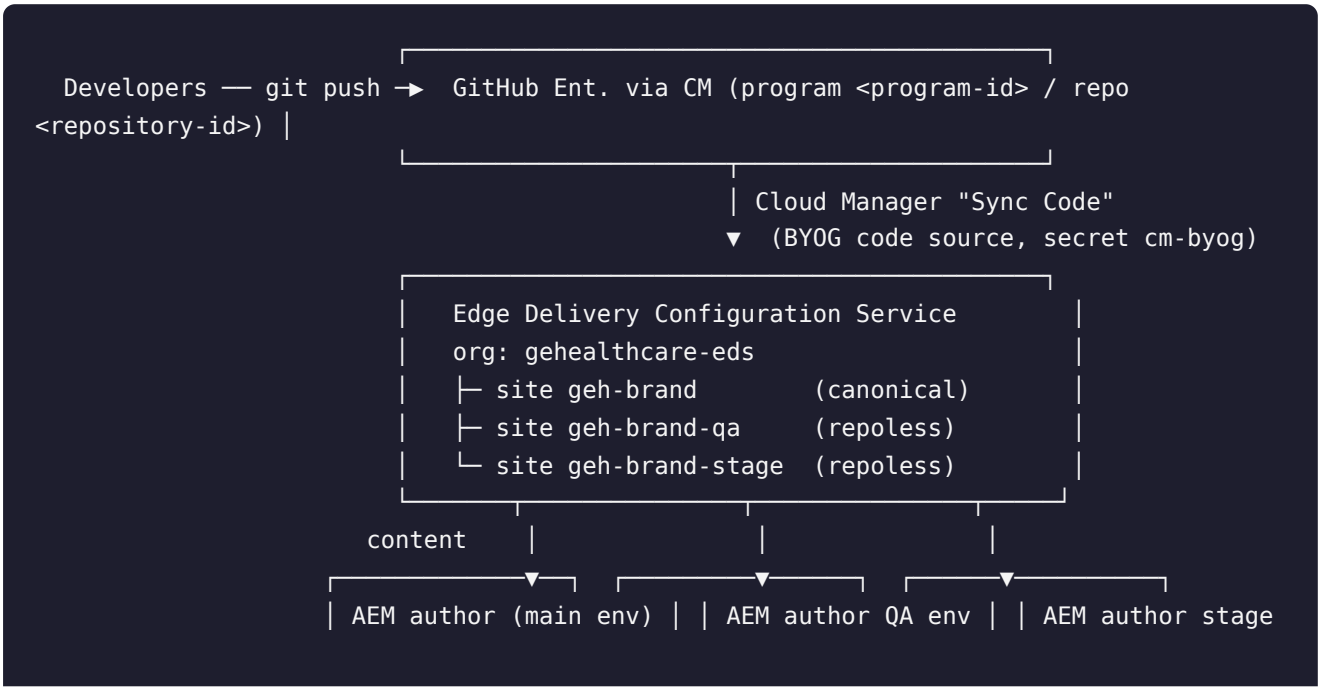
Official references used throughout (all screenshots in this guide come from these pages):

- Developer tutorial (AEM authoring / Universal Editor): https://www.aem.live/developer/ue-tutorial
- Bring Your Own Git: https://www.aem.live/developer/byo-git
- Repoless — one codebase, many sites: https://www.aem.live/docs/repoless
- Configuration Service setup (Admin API): https://www.aem.live/docs/config-service-setup
- Admin API keys: https://www.aem.live/docs/admin-apikeys

- Cloud Manager — Manage repositories: https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/using-cloud-manager/managing-code/managing-repositories
- Cloud Manager — Add external repositories (GitHub Enterprise Server): https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/using-cloud-manager/managing-code/external-repositories
- Cloud Manager — Add an Edge Delivery site: https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/using-cloud-manager/edge-delivery-sites/add-edge-delivery-site
- Cloud Manager — Configure an Edge Delivery site to use an external Git repository: https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/using-cloud-manager/edge-delivery-sites/config-edge-delivery-site-with-byog

> **Security:** never paste an Admin API token or Cloud Manager secret into chat tools, tickets or Git. Keep them in a password manager or an environment variable (`AEM_TOKEN`) and rotate them regularly.

How the pieces fit together



```
env │
                └──────────────┘  └──────────────┘  └──────────────┘
```

- **Code** lives once, in the private GitHub Enterprise repository (`github.gam.gehealthcare.com`) onboarded in Cloud Manager. Every site points to the same program id + repository id.
- **Content** is configured per site: each site's `content.source.url` points to the `franklin.delivery` endpoint of its own AEM author environment.
- **Site URLs** follow `https://<branch>--<site>--<org>.aem.page` (preview) and `.aem.live` (live), e.g. `https://main--geh-brand-qa--gehealthcare-eds.aem.live/`.

Prerequisites

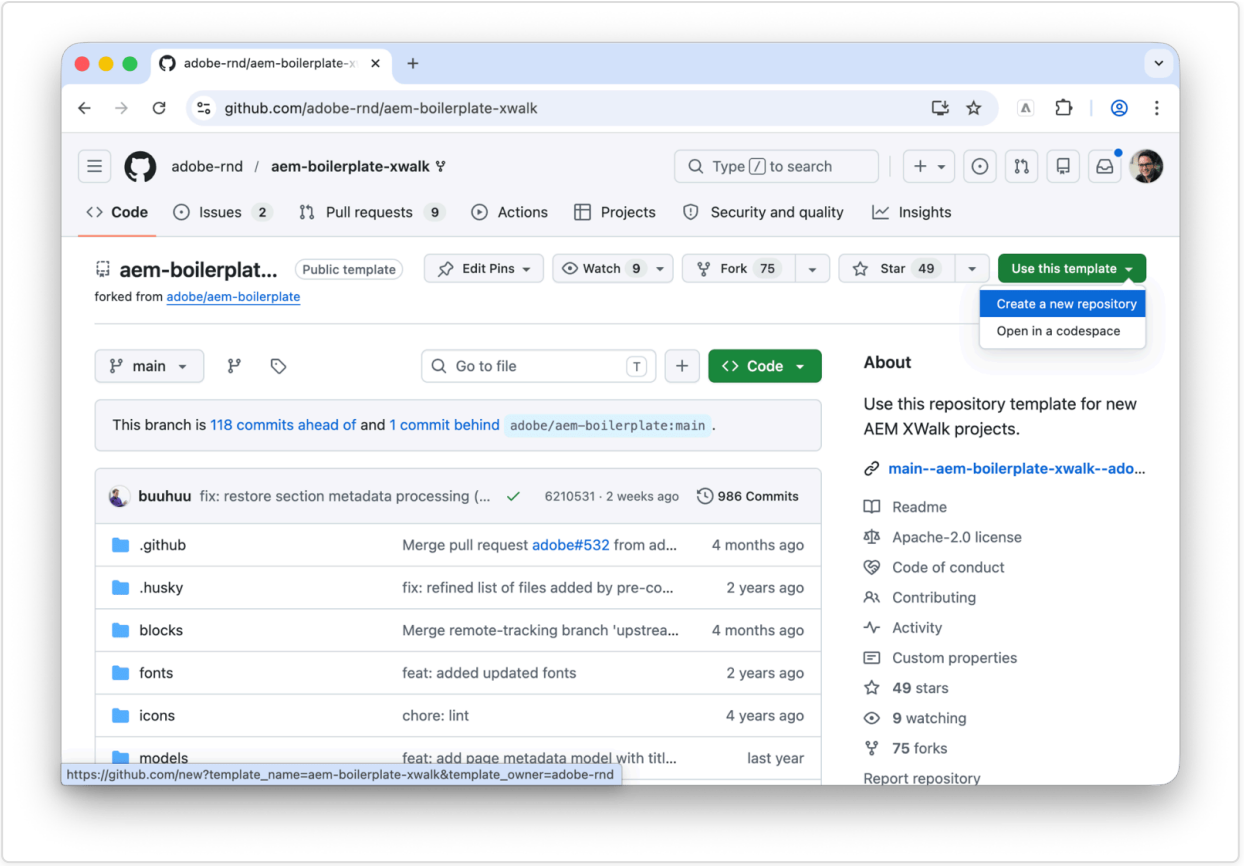
- A **github.com organization named** `gehealthcare-eds` that GE controls. Even when the code lives in Cloud Manager, the aem.live org name must exist on github.com so nobody else can claim it.
- A GitHub account with Git basics, Node.js/npm, and the AEM Sidekick extension.
- Access to the AEM as a Cloud Service author environments and to **Cloud Manager** for program `<program-id>` (a role that can manage repositories and Edge Delivery sites, e.g. Deployment Manager / Business Owner).
- An **Admin API token** with admin rights on the org/site (see Part D).

Part A — Create the canonical site from the boilerplate (GitHub)

This bootstraps the org and the first site in the configuration service. It follows the official developer tutorial.

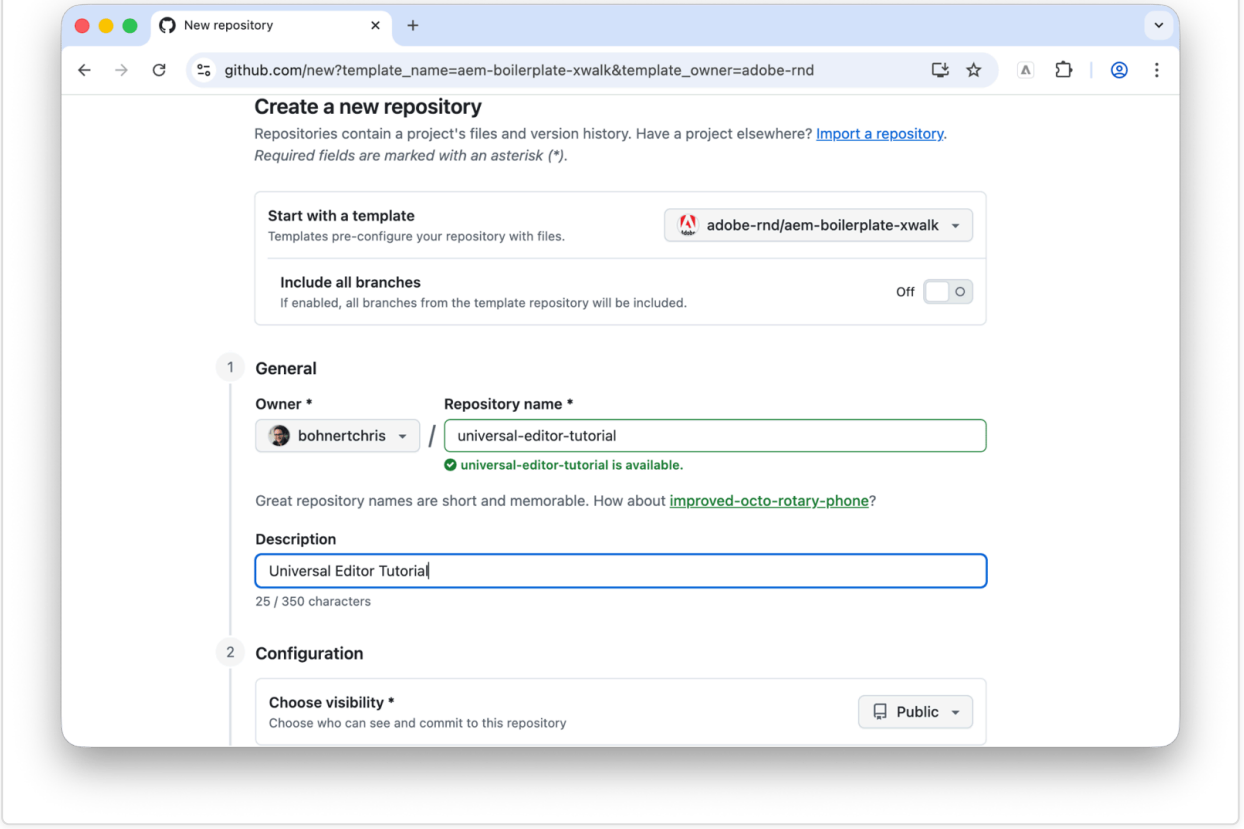
A1. Create the repository from the xwalk boilerplate

1. Sign in to github.com and open the AEM authoring boilerplate: https://github.com/adobe-rnd/aem-boilerplate-xwalk.
2. Click **Use this template → Create a new repository**.



Source: aem.live developer tutorial

3. Choose owner **`gehealthcare-eds`**, name the repository **`geh-brand`**, add a description and click **Create repository**.

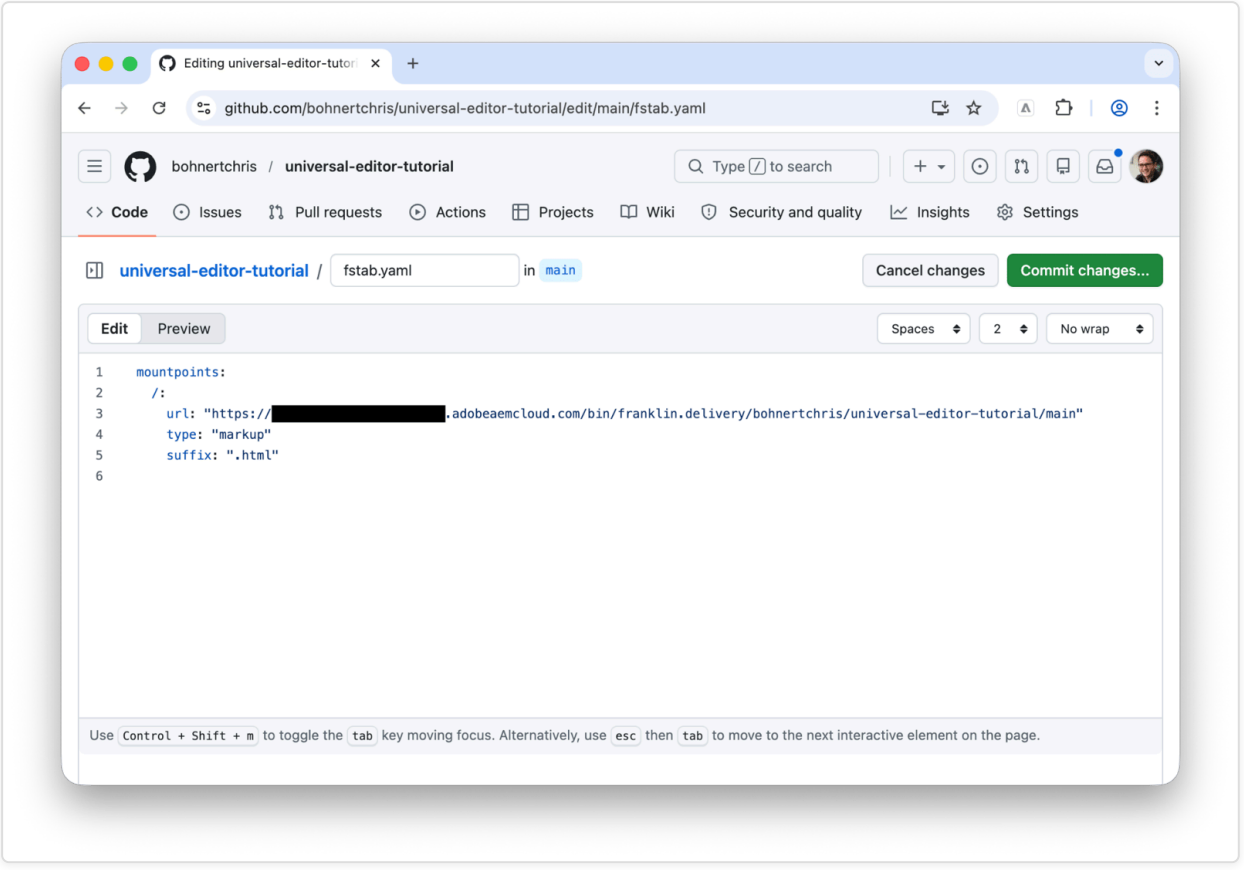


Source: aem.live developer tutorial

A2. Point `fstab.yaml` at the AEM author

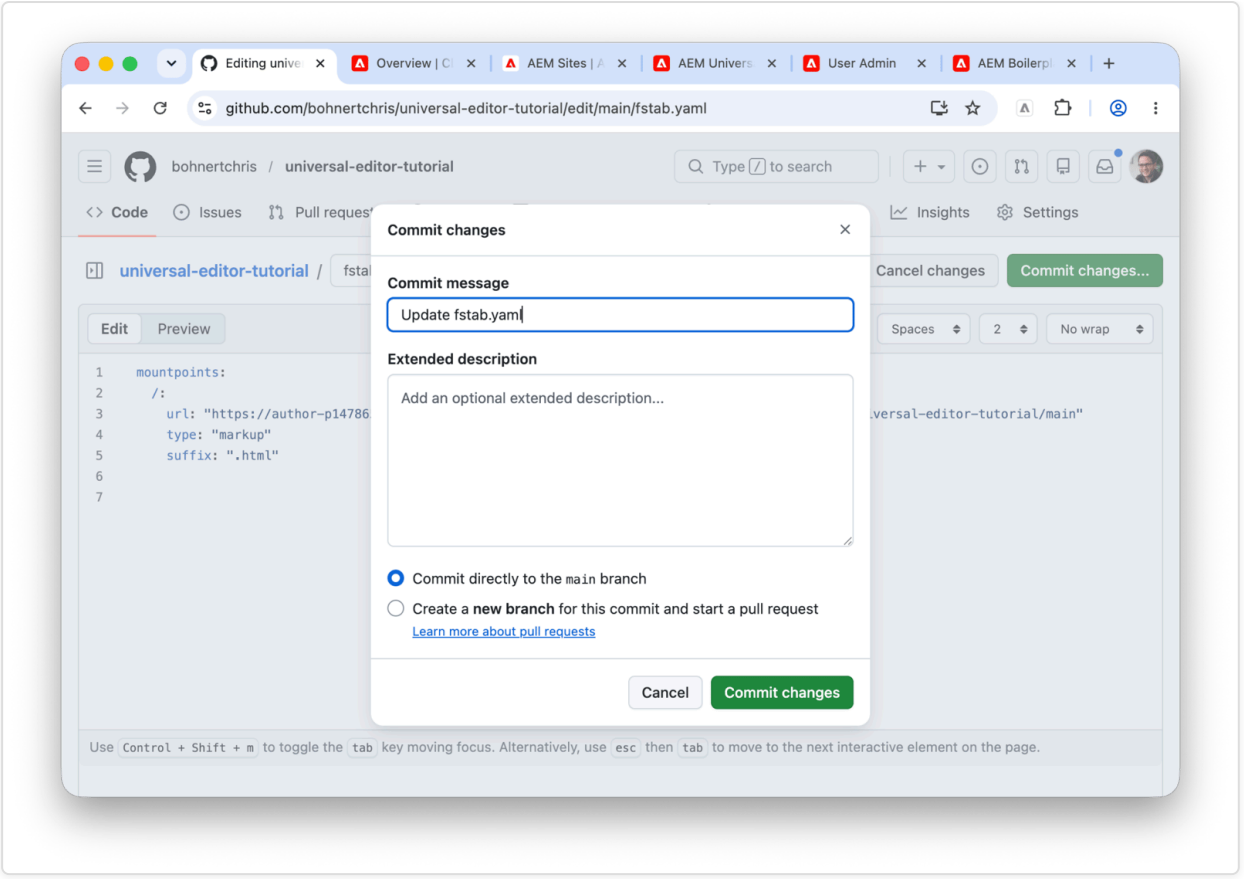
1. In the new repository open **`fstab.yaml`** and click the pencil (**Edit this file**).
2. Replace the mount point with the AEM author `franklin.delivery` URL (format `https://<aem-author>/bin/franklin.delivery/<owner>/<repository>/main`):

```
mountpoints:
  /:
    url: "https://author-p<program-id>-e<env-
id>.adobeaemcloud.com/bin/franklin.delivery/gehealthcare-eds/geh-brand/main"
    type: "markup"
    suffix: ".html"
```



Source: aem.live developer tutorial

3. Click **Commit changes…**, add a message and commit **directly to `main`**.

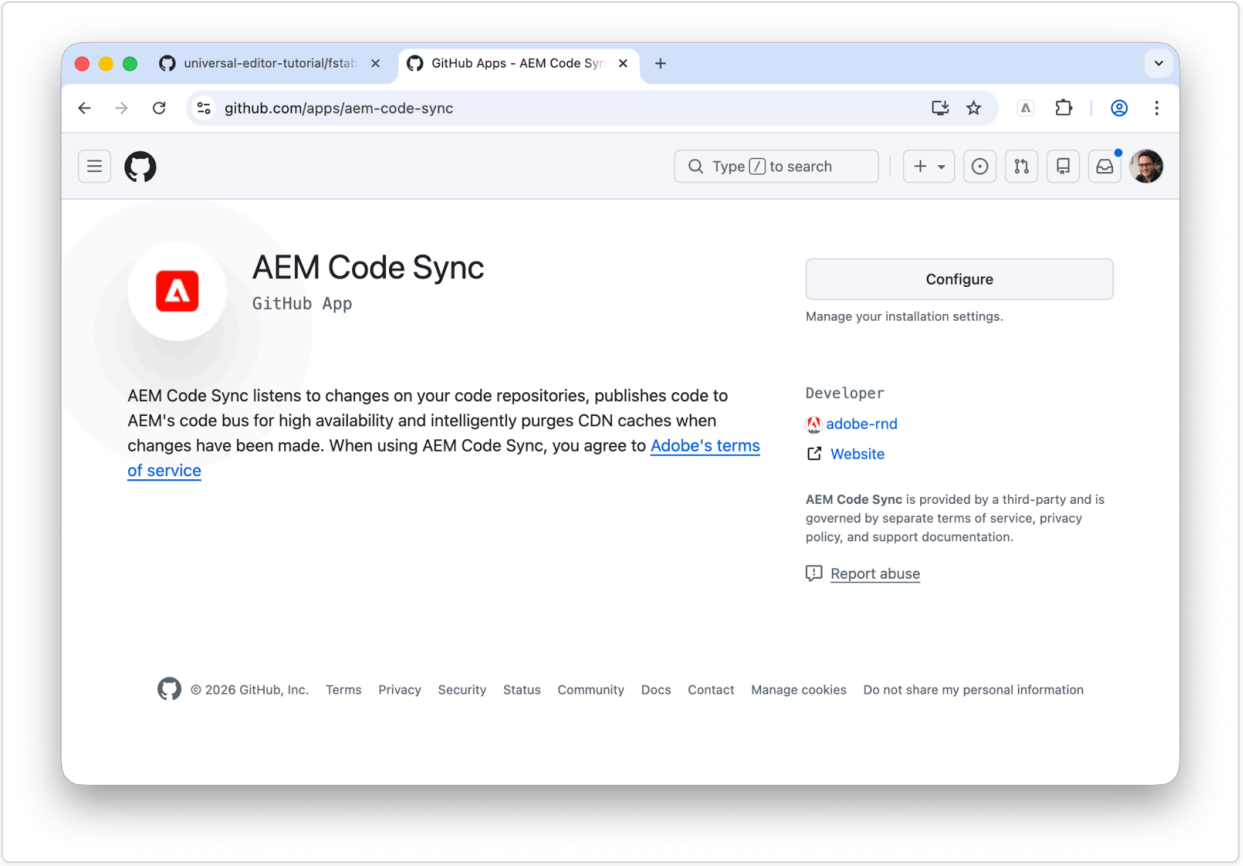


Source: aem.live developer tutorial

> Update `fstab.yaml` **before** installing AEM Code Sync. Once Code Sync has written the site into the configuration service, `fstab.yaml` is no longer read; later content-source changes are made in the configuration service (Site Admin tool or Admin API — see Part E).

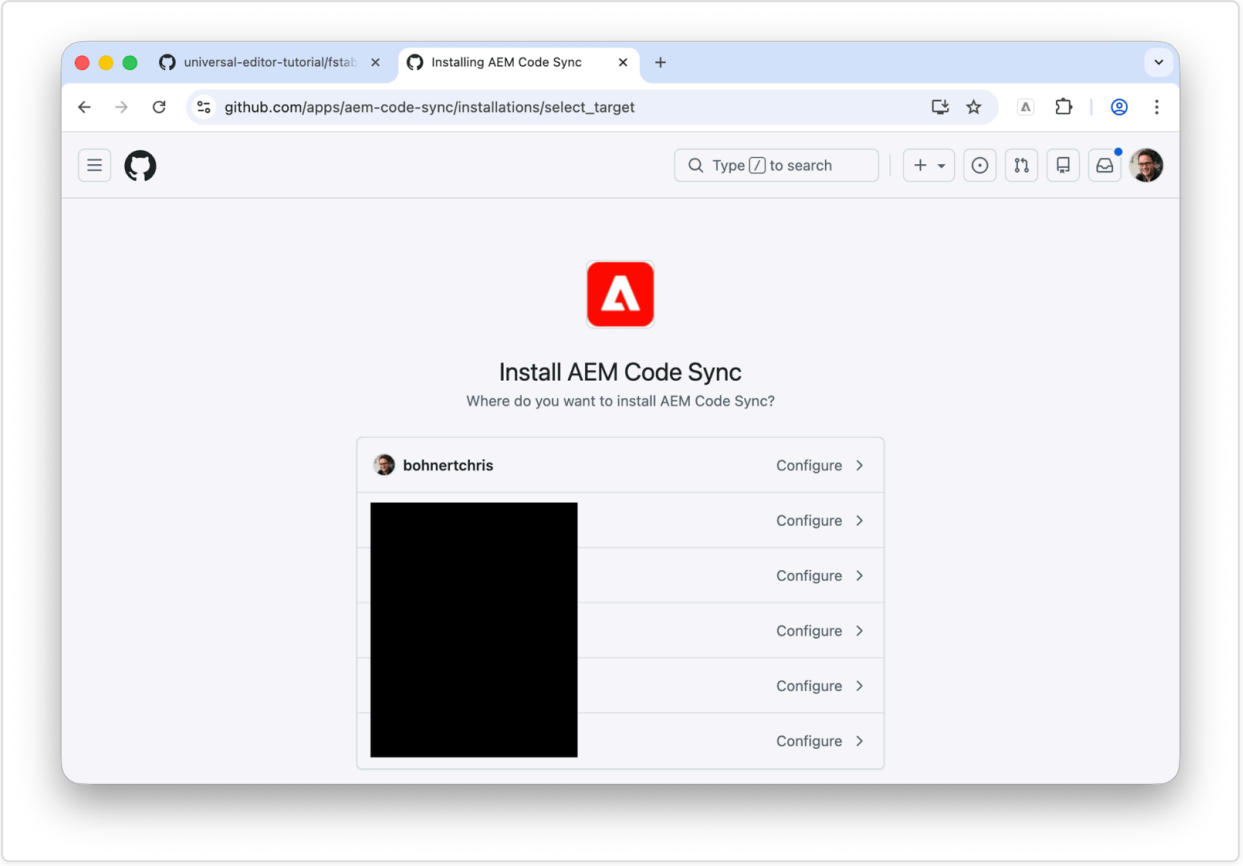
A3. Install AEM Code Sync

1. In the same browser open https://github.com/apps/aem-code-sync and click **Configure**.



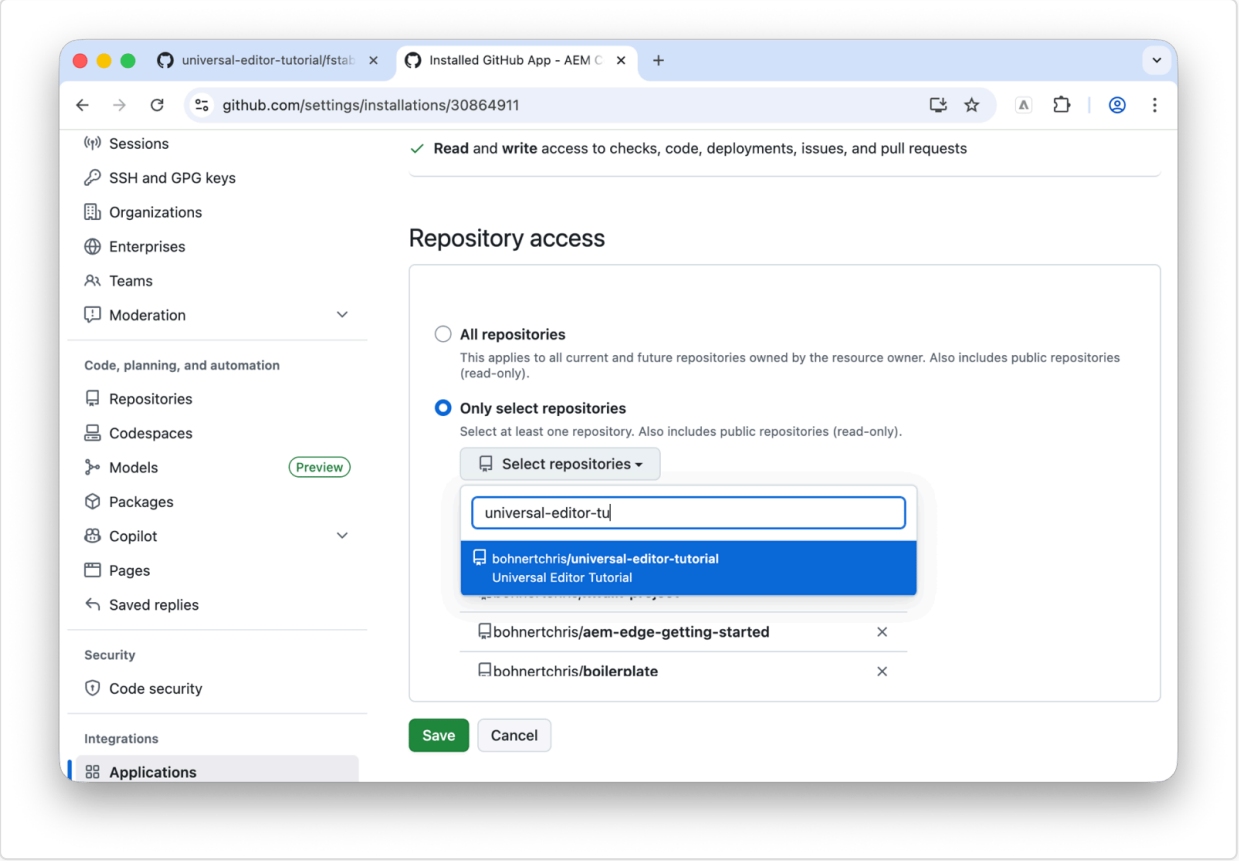
Source: aem.live developer tutorial

2. Pick the **`gehealthcare-eds`** organization.



Source: aem.live developer tutorial

3. Under **Repository access** choose **Only select repositories**, select `geh-brand` and **Save**.

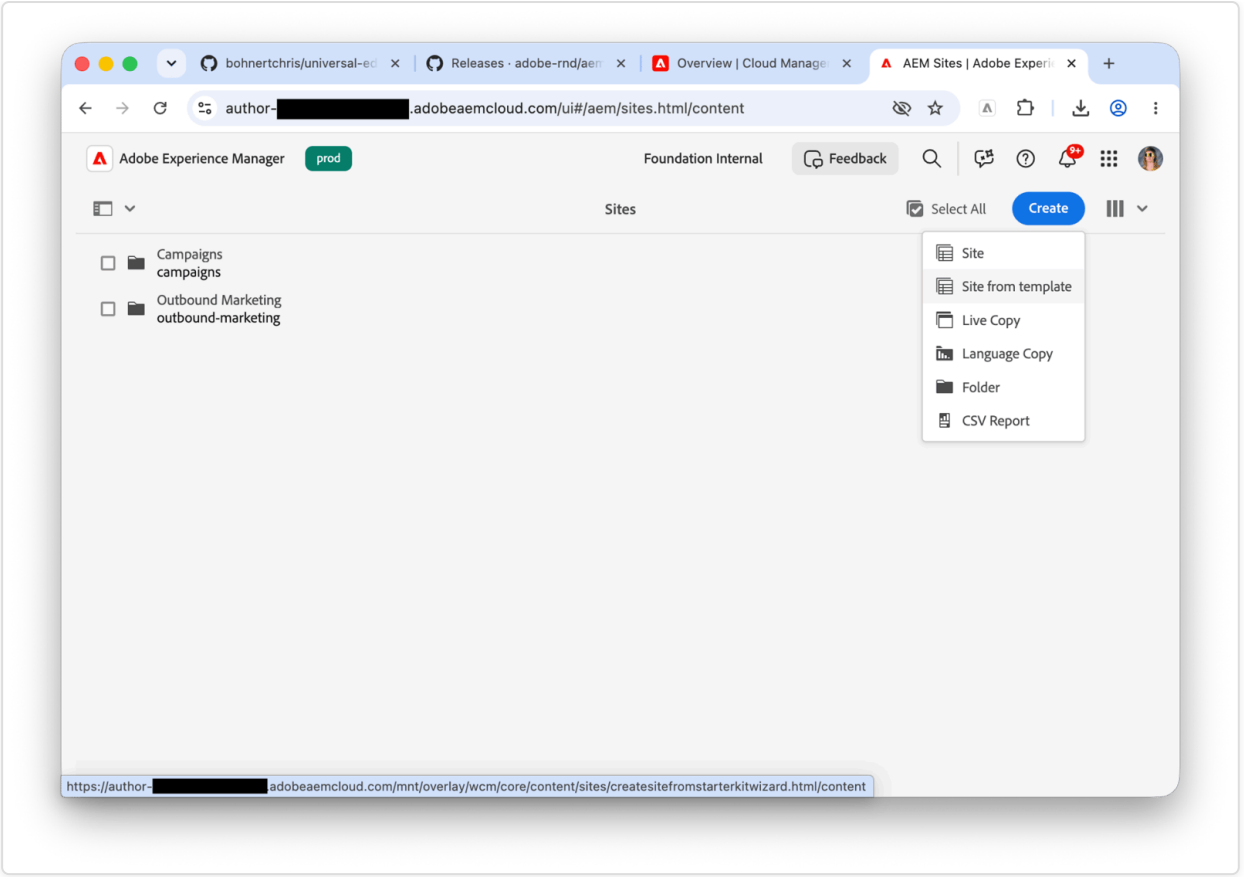


Source: aem.live developer tutorial

4. Complete the **AEM Configuration Setup** wizard that opens:
   - **Code** → Next.
   - **Content** → type is pre-set to *Other (bring your own markup)* and the URL is read from `fstab.yaml` → Next.
   - **Users** → add the admin users for this site configuration (your own e-mail first!) → Next.
   - **Finish** → review and **Save**.
5. Verify the code is served: https://main--geh-brand--gehealthcare-eds.aem.page/scripts/aem.js should return JavaScript.

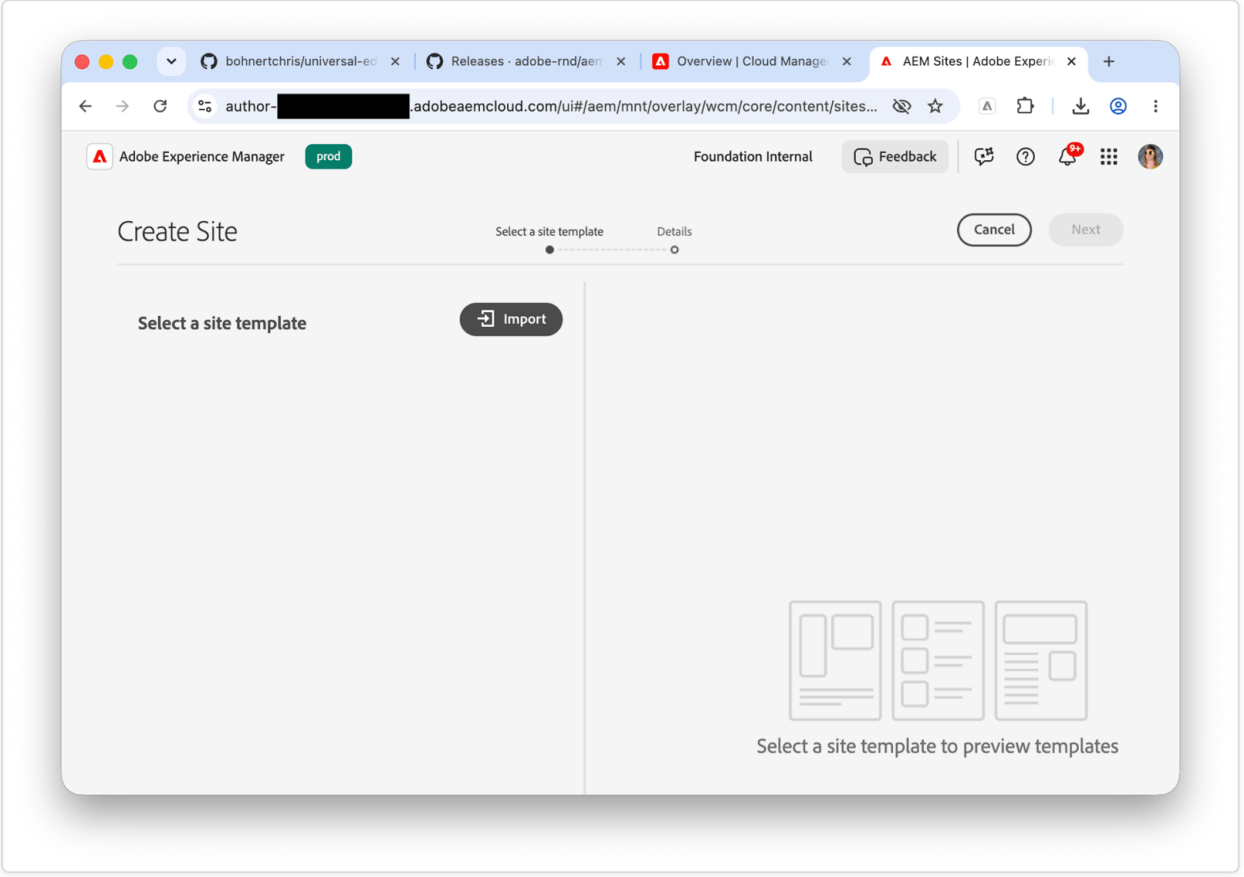
A4. Create the AEM site from the template

1. Download the latest site template (`.zip`) from https://github.com/adobe-rnd/aem-boilerplate-xwalk/releases.
2. In the AEM author **Sites** console click **Create → Site from template**.



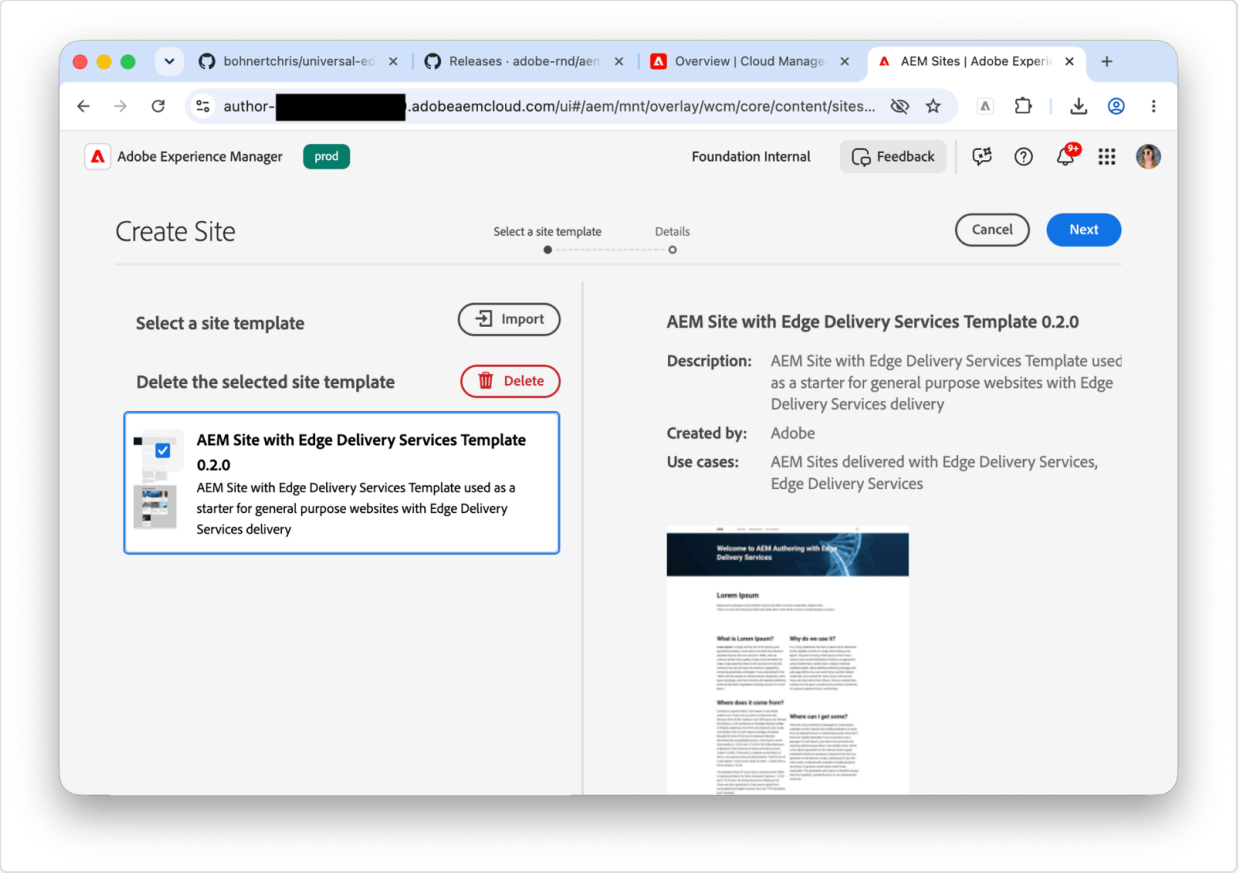
Source: aem.live developer tutorial

3. Click **Import** and upload the template (only needed once per environment; it can be reused for more sites).



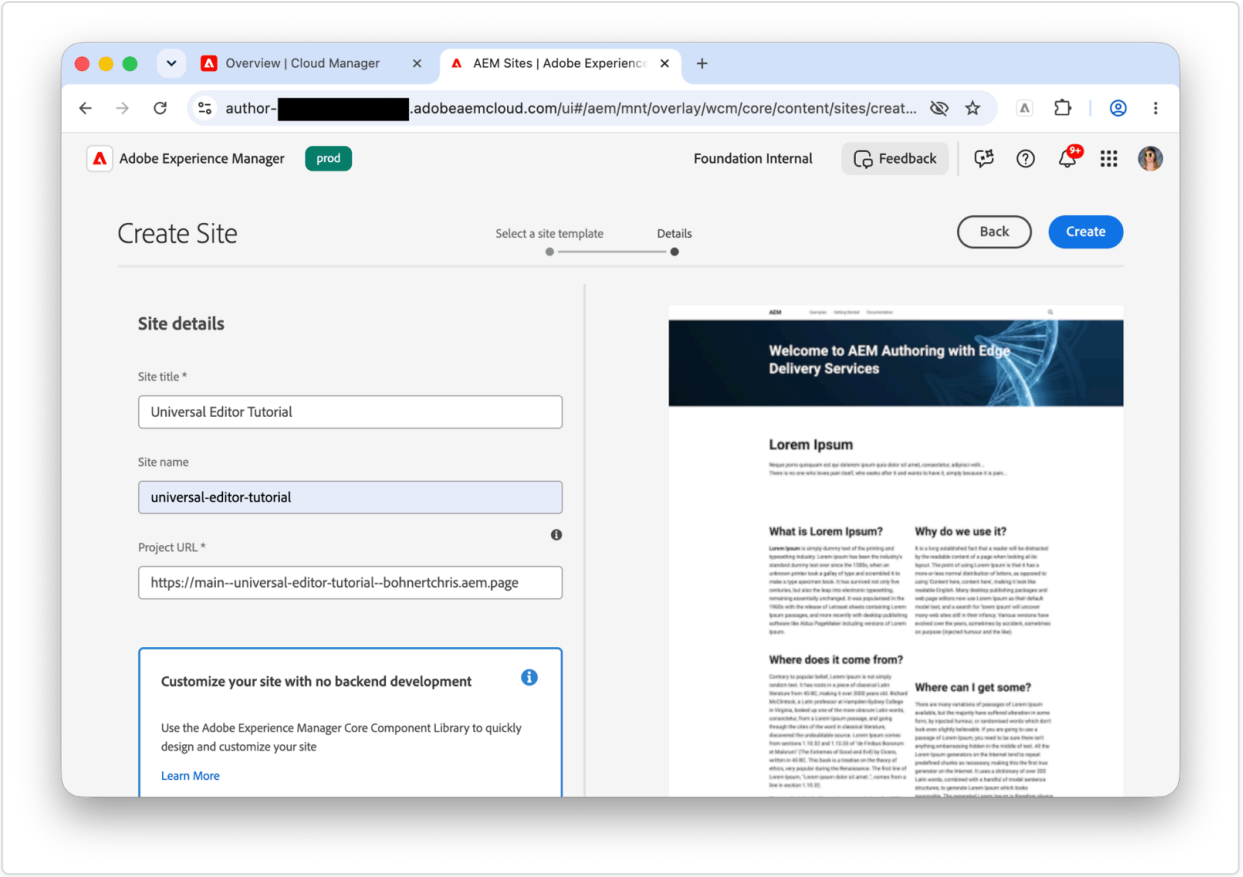
Source: aem.live developer tutorial

4. Select the imported template → **Next**.



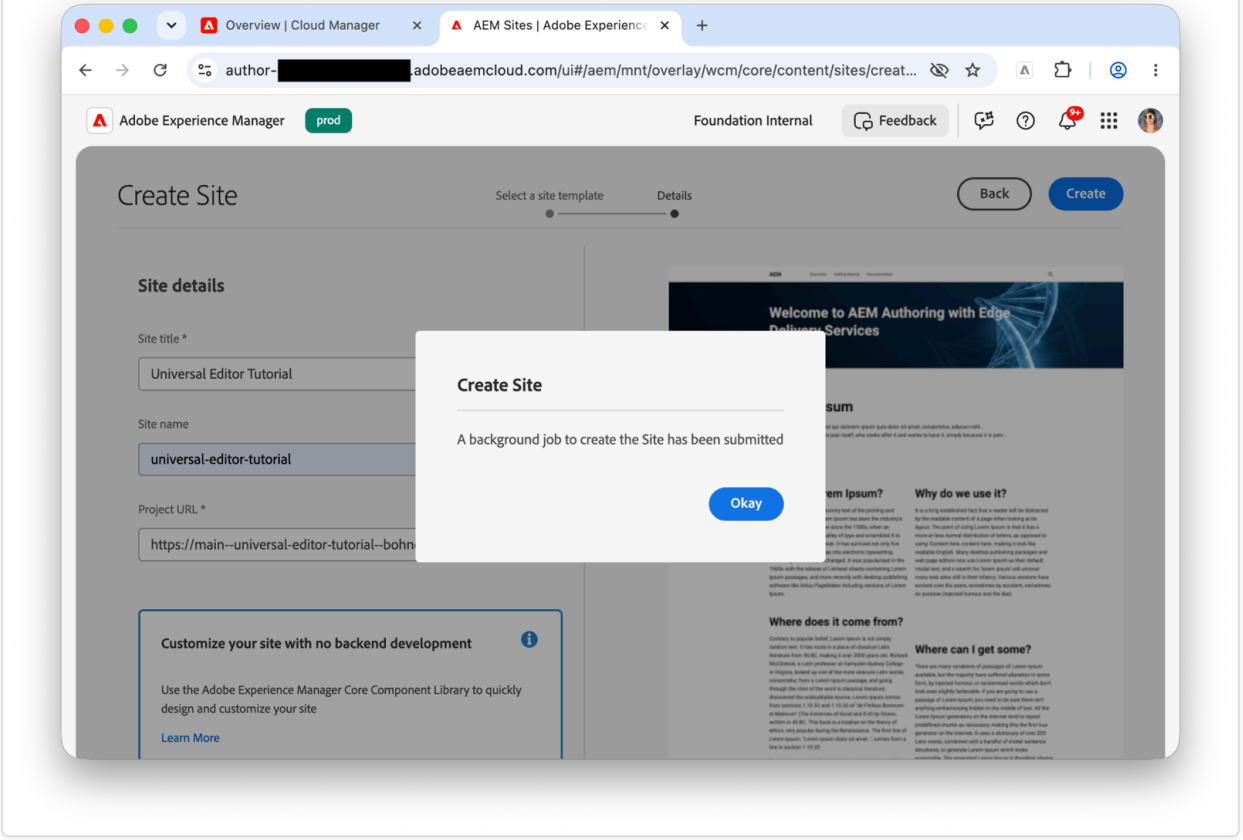
Source: aem.live developer tutorial

5. Fill in **Site title**, **Site name** (`geh-brand`) and **Project URL** `https://main--geh-brand--gehealthcare-eds.aem.page` , then **Create**.



Source: aem.live developer tutorial

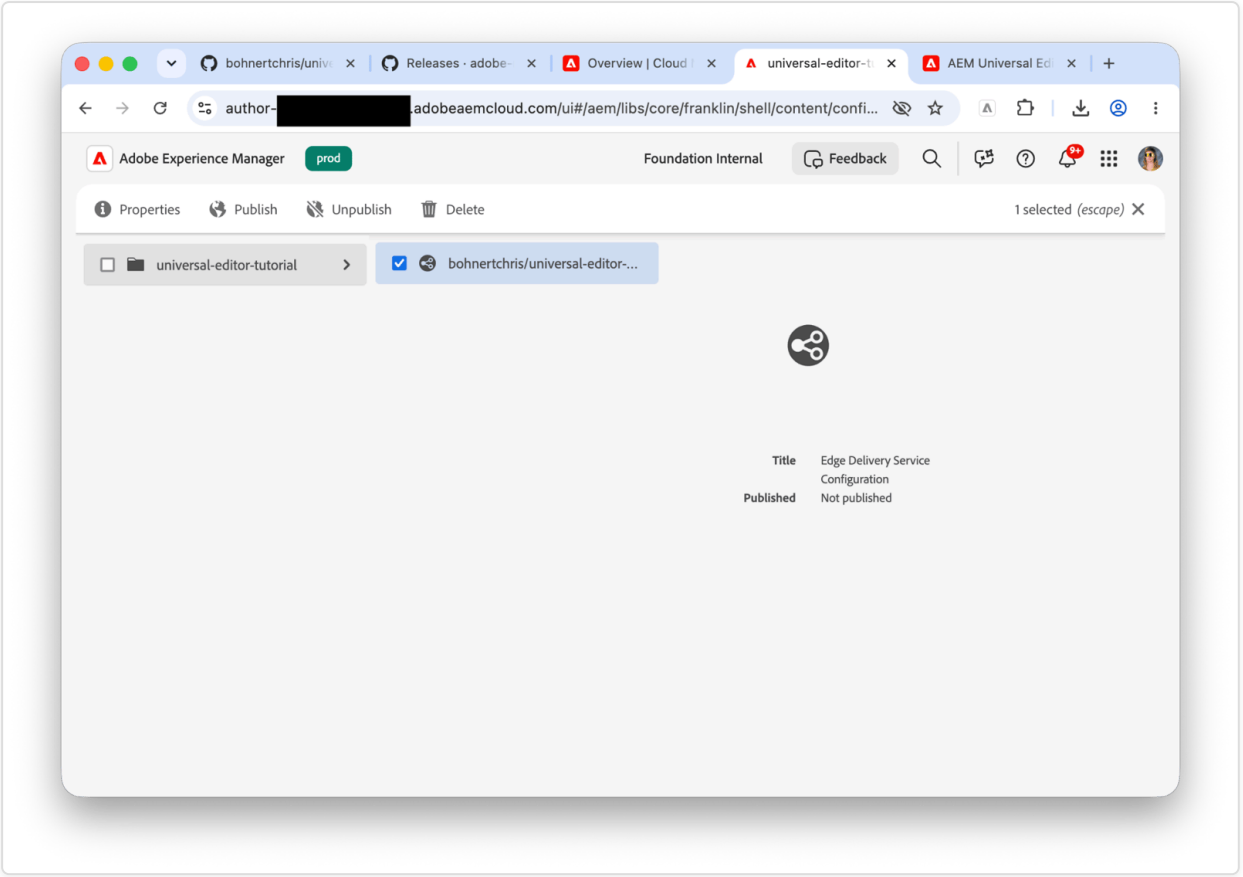
6. Confirm the dialog with **OK**.



Source: aem.live developer tutorial

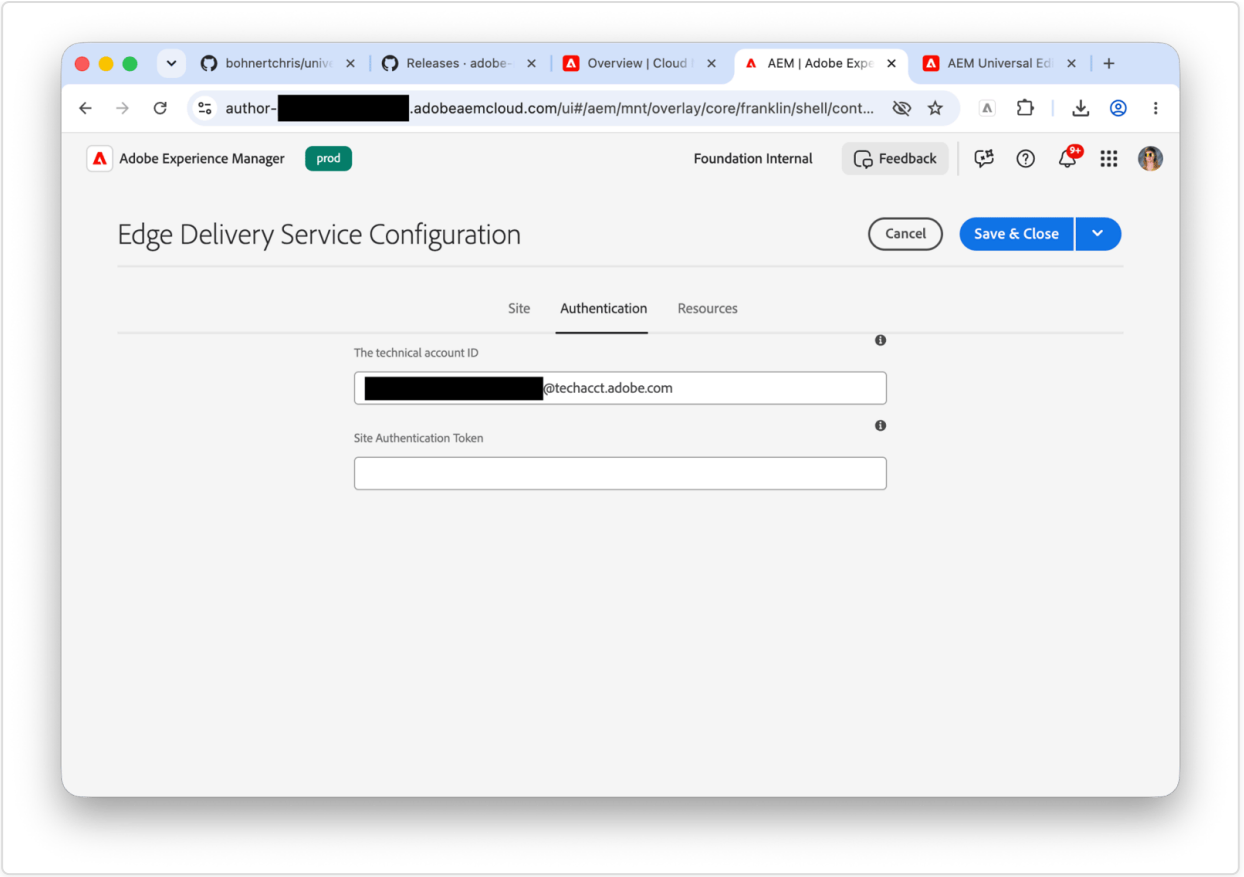
A5. Give the technical account publish rights

1. In AEM go to **Tools → Cloud Services → Edge Delivery Services Configuration**, select the configuration created for the site and click **Properties**.



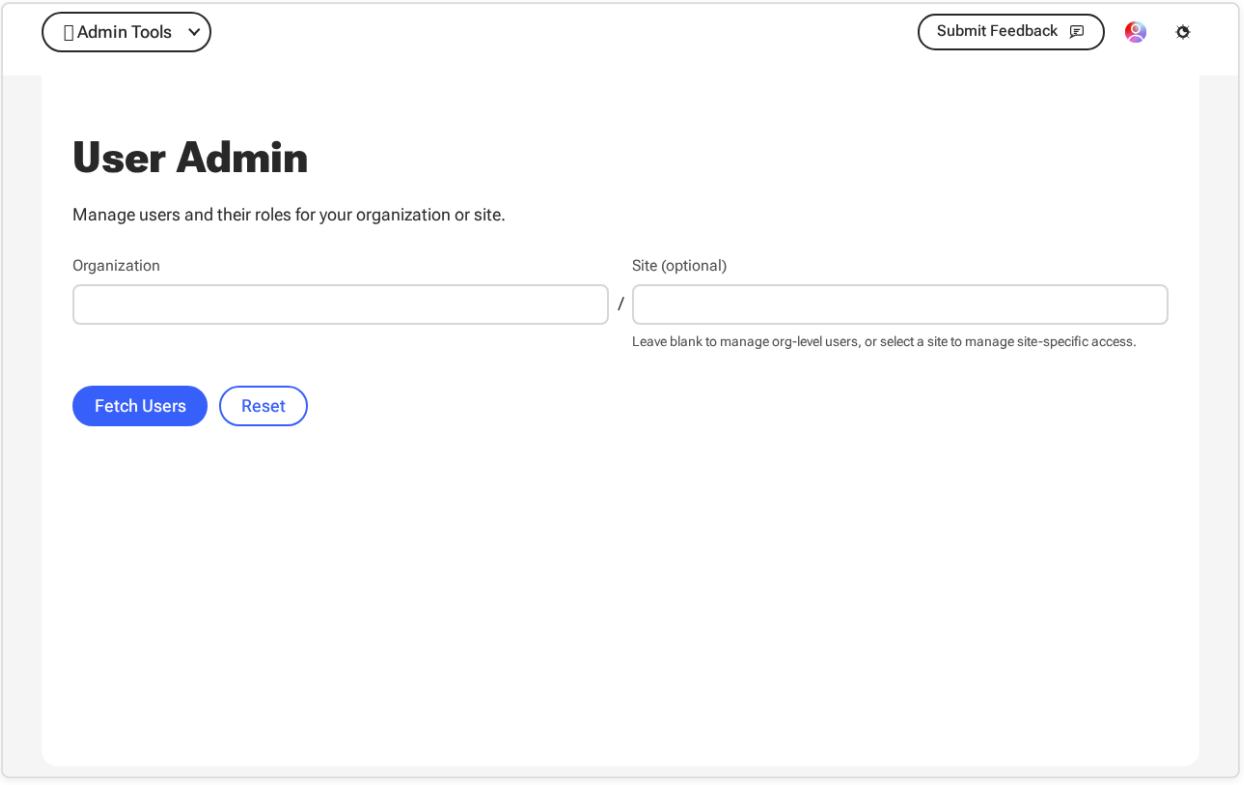
Source: aem.live developer tutorial

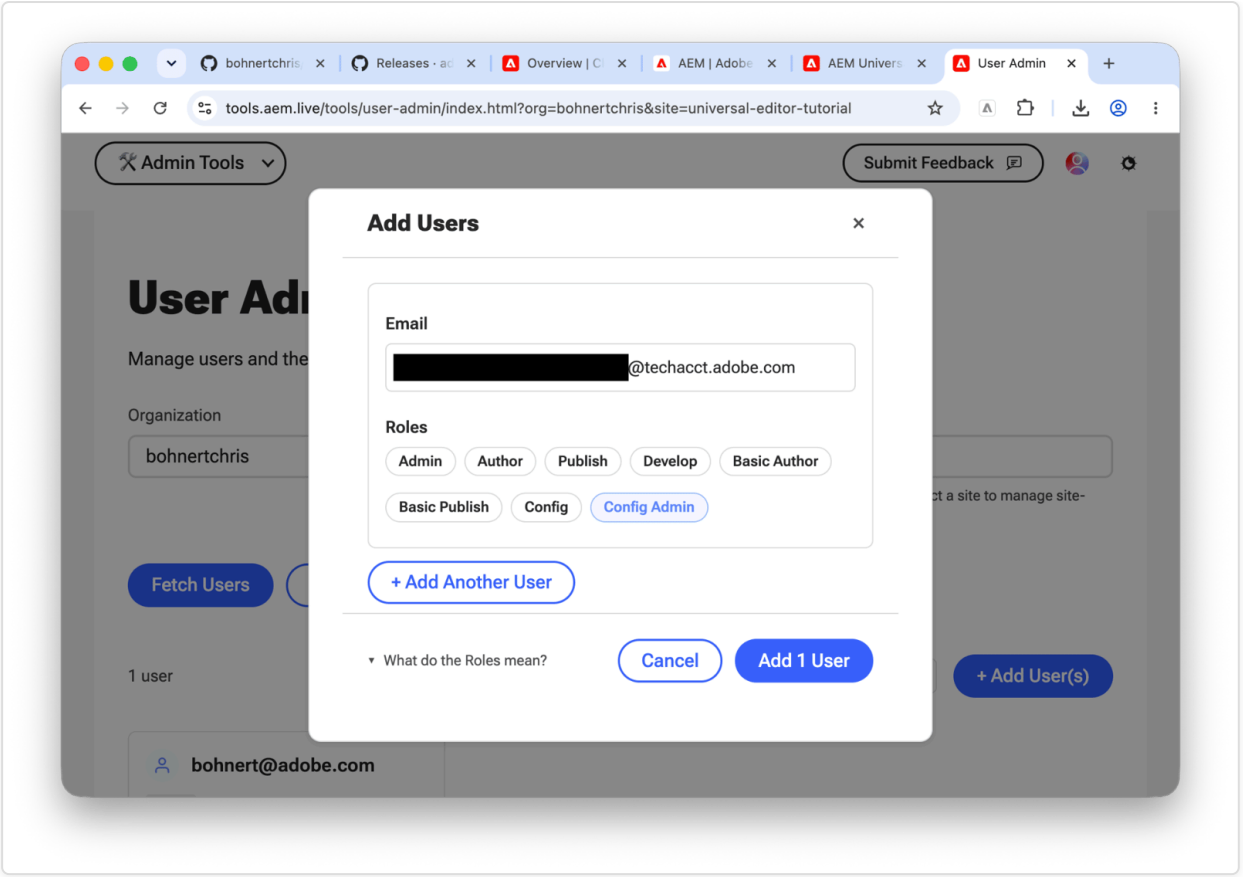
2. On the **Authentication** tab copy the **technical account ID** (`…@techacct.adobe.com`).



Source: aem.live developer tutorial

3. Open the **User Admin** tool https://tools.aem.live/tools/user-admin/index.html, load the site and **first make sure your own e-mail is listed as Admin** (otherwise you can lock yourself out). Then add the technical account with role **Config Admin**.





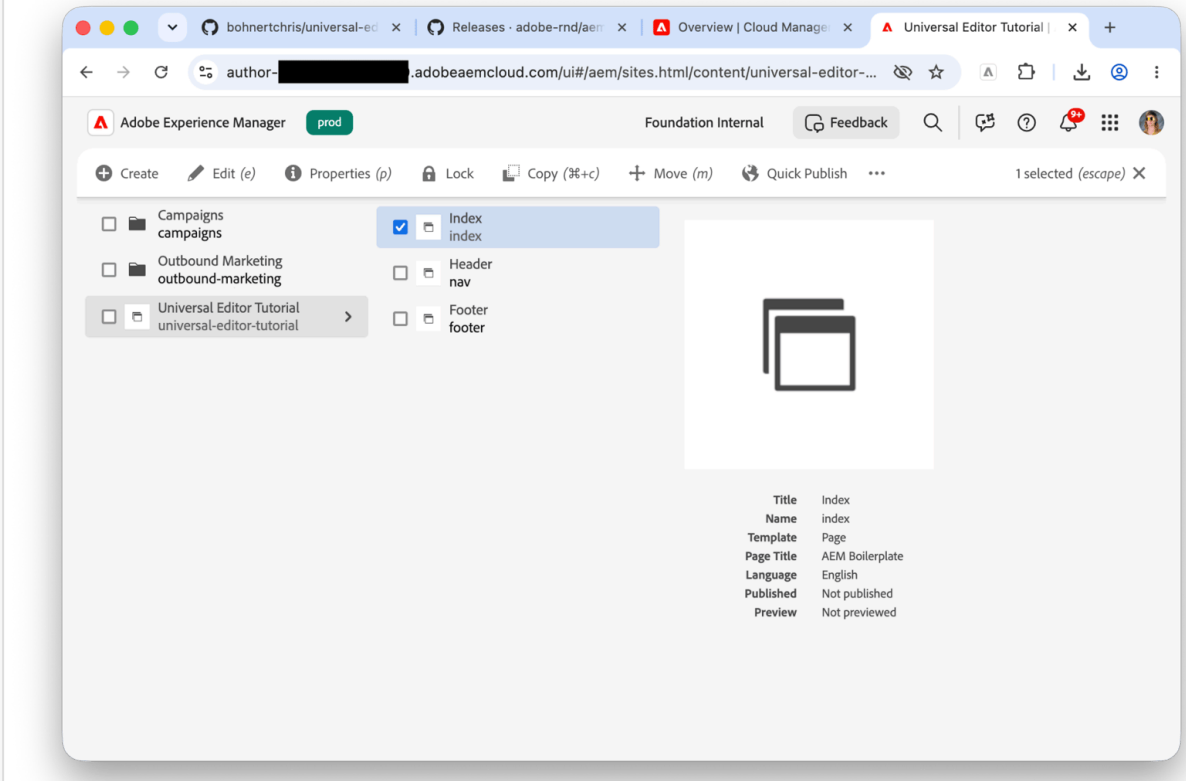
Source: aem.live developer tutorial

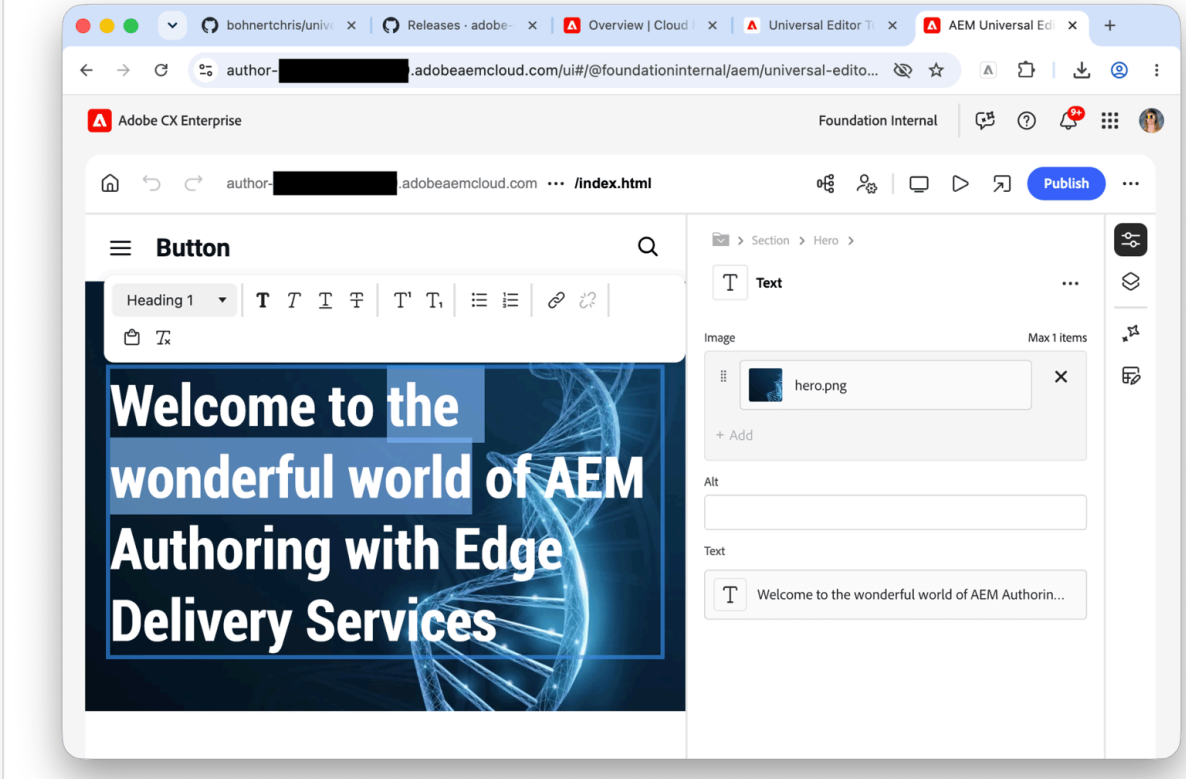
Allow a minute or two for the configuration to propagate.

> Repeat A4–A5 in **every AEM author environment** that will feed a site (each environment has its own technical account; that ID is the `config_admin` value used in Part E).

A6. Edit and publish a first page

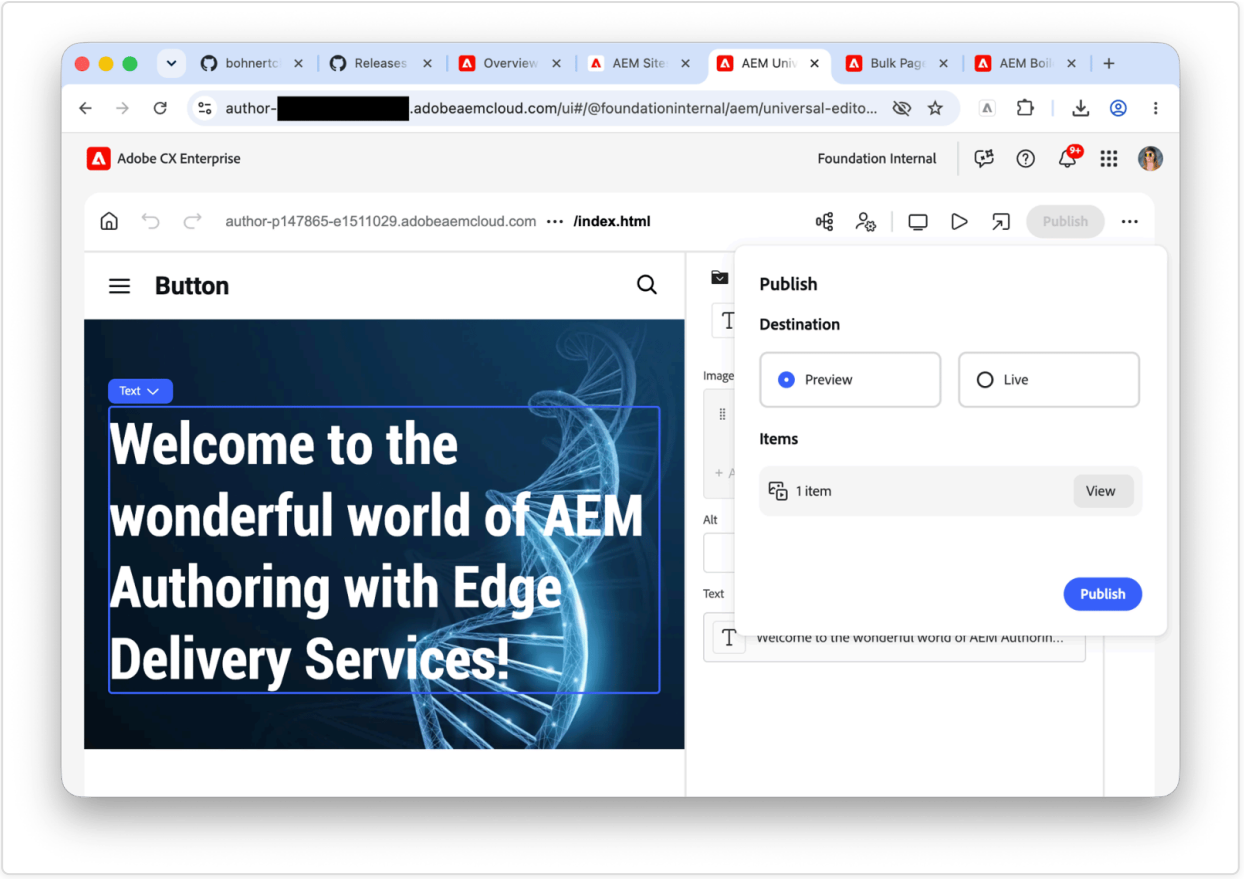
1. In the Sites console open `index` of the new site and click **Edit** — the Universal Editor opens (sign in with Adobe if asked).





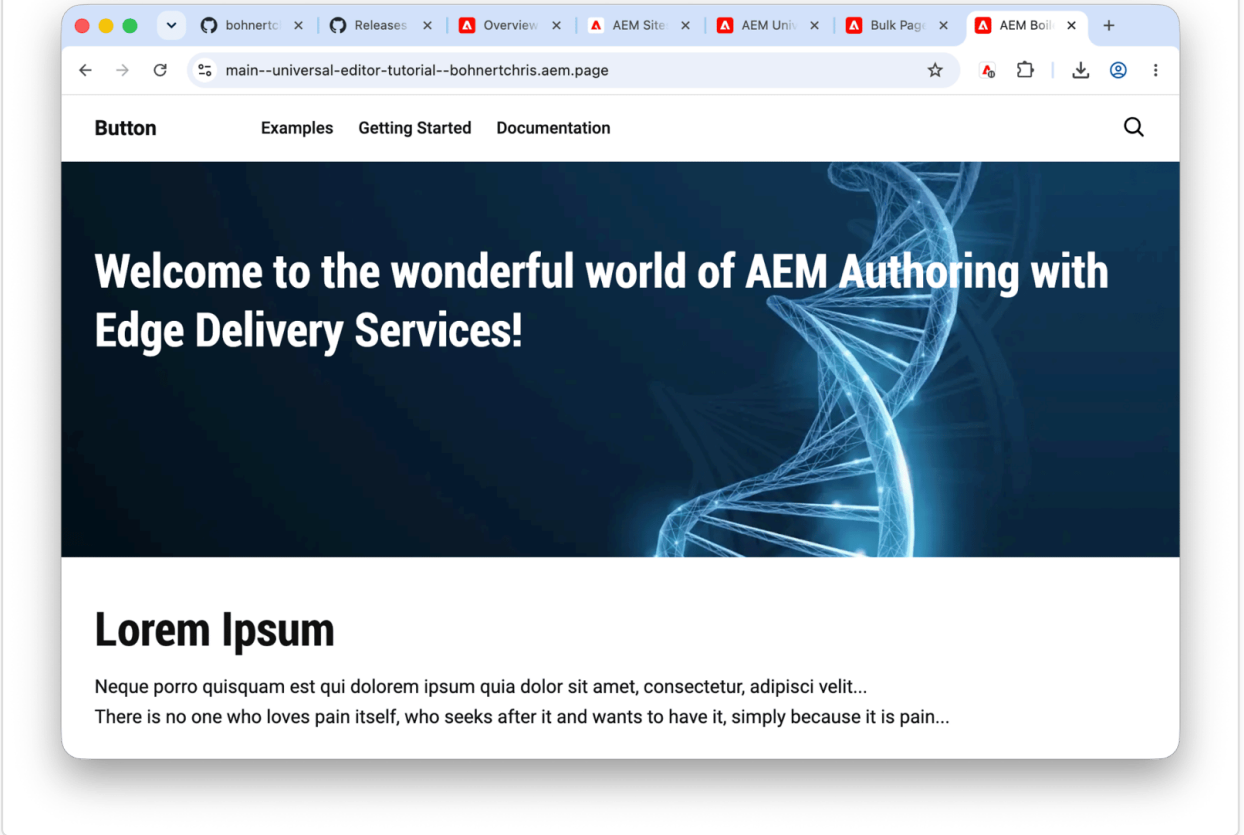
Source: aem.live developer tutorial

2. Click **Publish**, choose **Preview** as destination and publish.



Source: aem.live developer tutorial

3. Open https://main--geh-brand--gehealthcare-eds.aem.page/ — the page is served by Edge Delivery.



Source: aem.live developer tutorial

Part B — Onboard the GitHub Enterprise repository in Cloud Manager and connect it to the Edge Delivery site

In this setup the code lives in GE's **GitHub Enterprise Server** (self-hosted GitHub), e.g. `https://github.gam.gehealthcare.com`. Cloud Manager links it as a **Private Repository** with an **external Git URL**, and Edge Delivery pulls the code from it through **Bring your own Git**.

Part B has three stages:

1. **Onboard the GitHub Enterprise repository in Cloud Manager** (B1–B4): push the code, add it as a Private Repository, validate ownership with an access token, configure the webhook.
2. **Use the repository in the Edge Delivery site** (B5–B7): add the site in Cloud Manager, choose **Bring your own Git**, and point the site's code source at the repository.
3. **Sync the code** (B8–B9): **Sync Code** asks for a branch; after the first sync, every commit syncs automatically.

Before you start you need:

- the **Deployment Manager** or **Business Owner** role in Cloud Manager;
- a repository on the GitHub Enterprise server, e.g. `https://github.gam.gehealthcare.com/<org>/geh-brand`, and a user who can create a **personal access token (PAT)** and **webhooks** on it;
- an **API key** from an Adobe Developer Console integration project (used in the webhook URL, B3).

B1. Push the code to the GitHub Enterprise repository

Create an empty repository `geh-brand` on the GitHub Enterprise server, then push the code created in Part A to it:

```
git clone https://github.com/gehealthcare-eds/geh-brand.git
cd geh-brand
```

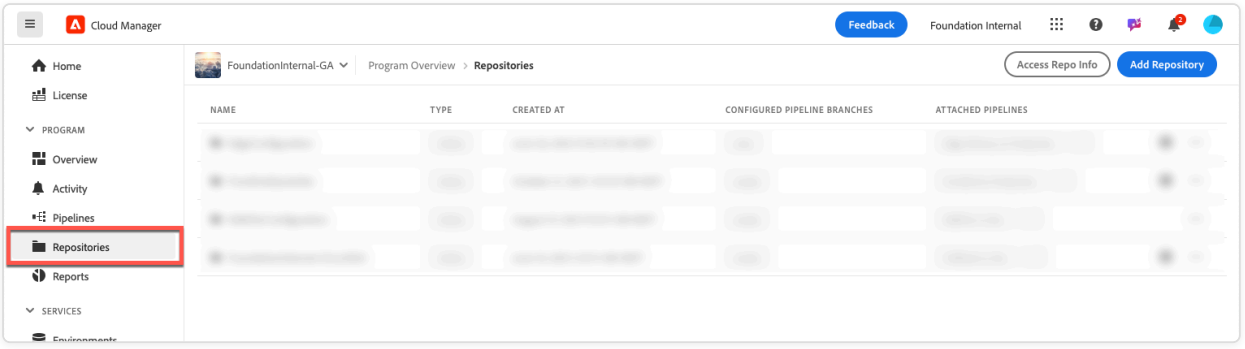
```
git remote add ghe https://github.gam.gehealthcare.com/<org>/geh-brand.git
git push ghe main          # Edge Delivery requires a "main" branch
```

From now on developers commit and push to the GitHub Enterprise repository (`ghe` remote).

B2. Add the repository in Cloud Manager as a Private Repository (external Git URL)

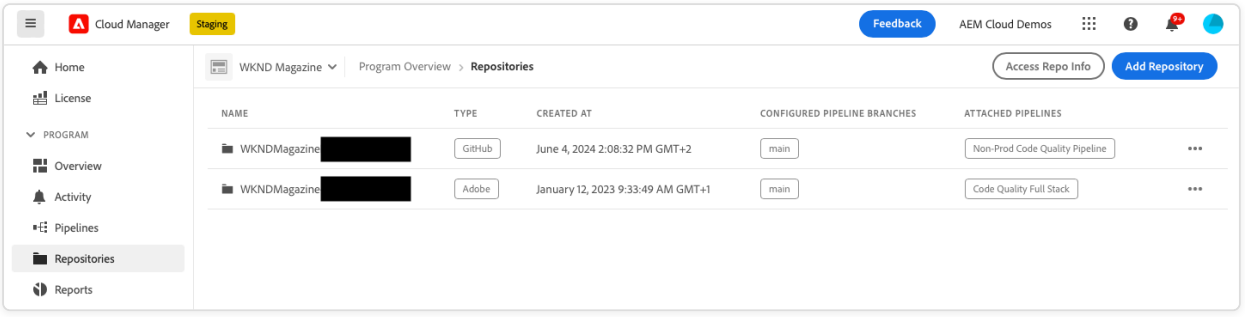
1. Sign in at https://experience.adobe.com → **Experience Manager** → **Cloud Manager** (or go directly to https://my.cloudmanager.adobe.com), and select the GE organization near the upper-right corner.

2. On the **My Programs** console open program **<program-id>**. In the side menu, under **Program**, click **Repositories**.



Source: Adobe Experience League — Add external repositories in Cloud Manager

3. Near the upper-right corner click **Add Repository**.

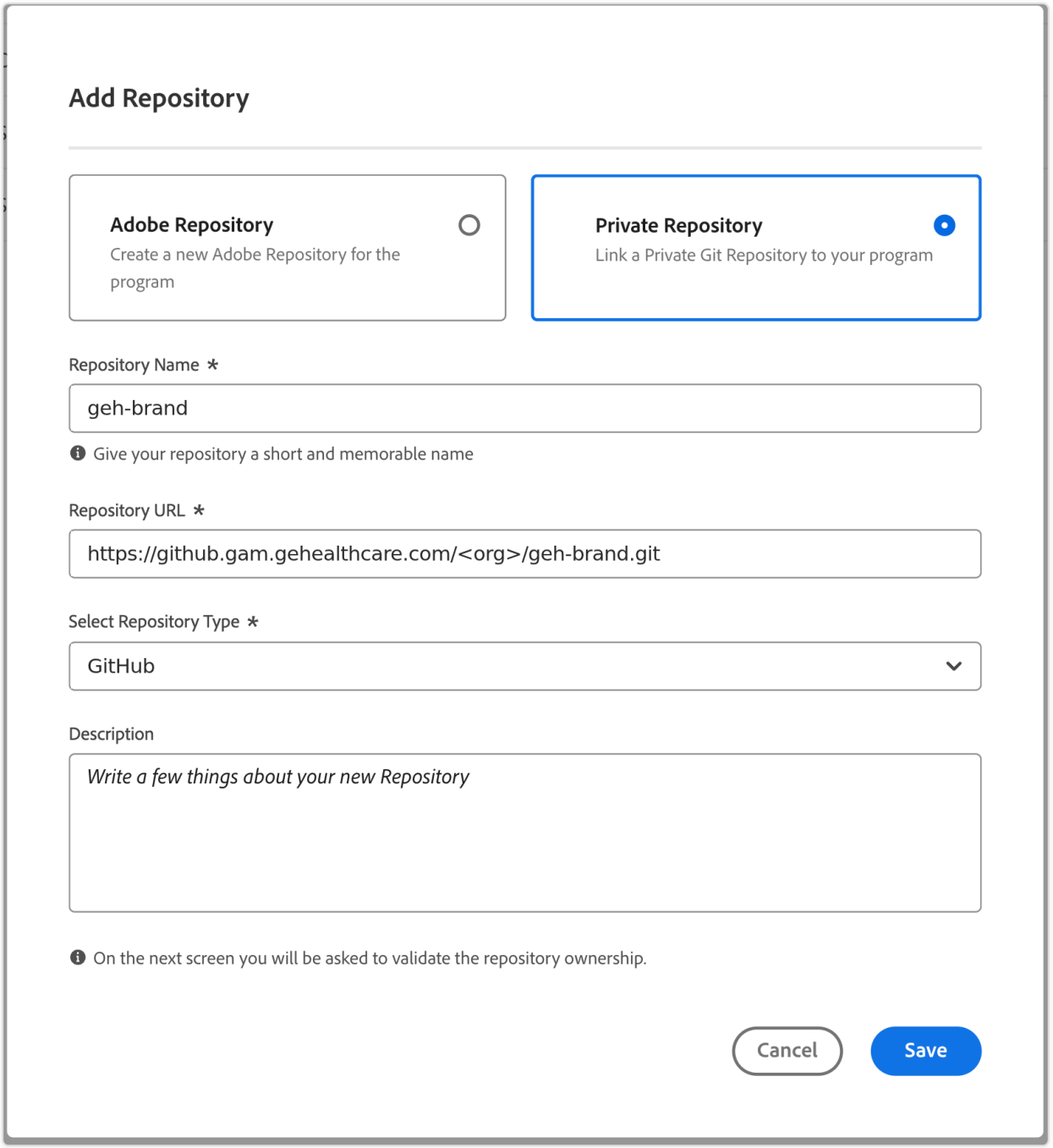


Source: Adobe Experience League — Add an Adobe repository in Cloud Manager

4. In the **Add Repository** dialog select **Private Repository** (*Link a Private Git Repository to your program*) and fill in:

Field	Value for GE
Repository Name	`geh-brand`

Field	Value for GE
Repository URL	the external GitHub Enterprise URL, ending in `.git`: `https://github.gam.gehealthcare.com/<org>/geh-brand.git`
Select Repository Type	**GitHub** (GitHub Enterprise Server)
Description	optional



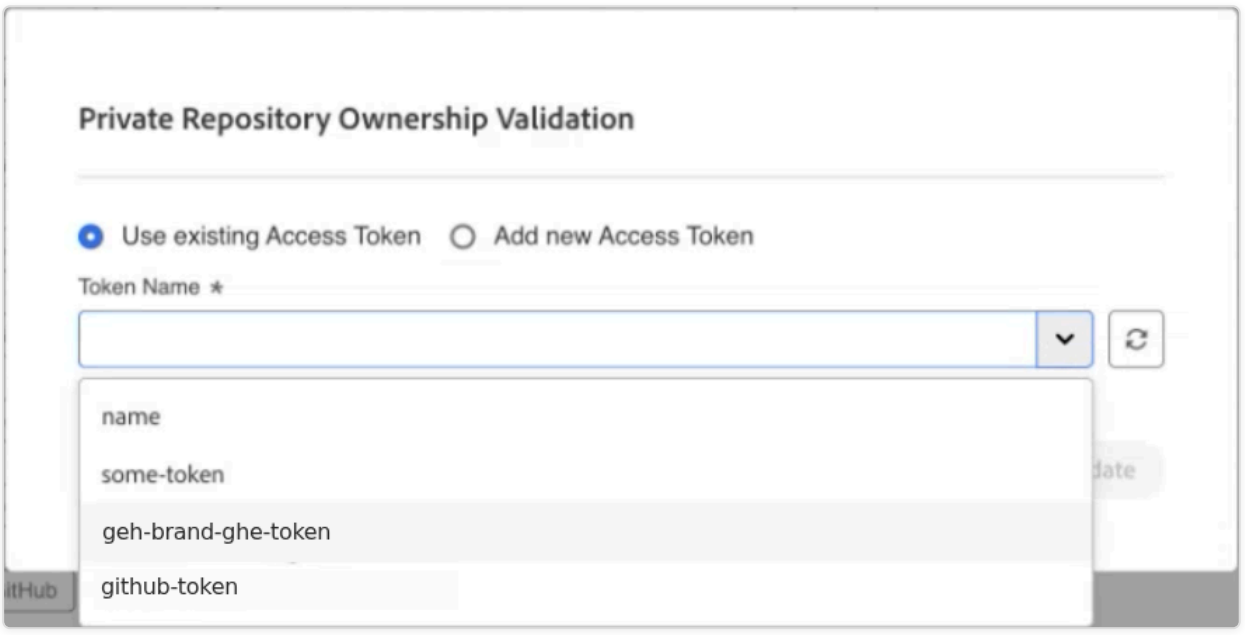
Source: Adobe Experience League — Add external repositories in Cloud Manager (field values adapted to the GE GitHub Enterprise repository)

5. Click **Save**. The **Private Repository Ownership Validation** dialog opens (B3).

> **Other options you can use.** The **Add Repository** dialog also offers an **Adobe Repository** (an Adobe-hosted repository created by Cloud Manager — Add an Adobe repository), a **Private Repository on github.com** (validated with the Cloud Manager GitHub app — Add a private GitHub repository), and private repositories on **GitLab, Bitbucket or Azure DevOps**. This guide uses the GitHub Enterprise URL only.

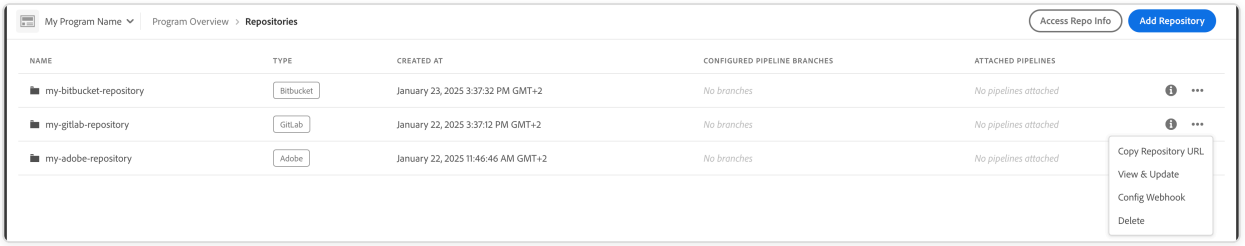
B3. Validate ownership with an access token and configure the webhook

1. On the GitHub Enterprise server create a **personal access token** for the repository with these permissions:
   - **Pull requests** — read and write
   - **Commit statuses** — read and write
   - **Repository metadata** — read-only
2. In the **Private Repository Ownership Validation** dialog choose **Add new Access Token**, enter a **Token Name** (e.g. `geh-brand-ghe`), paste the PAT into **Access Token** and click **Validate**. (If a token was already added for the organization, choose **Use existing Access Token** and select it in **Token Name**.)



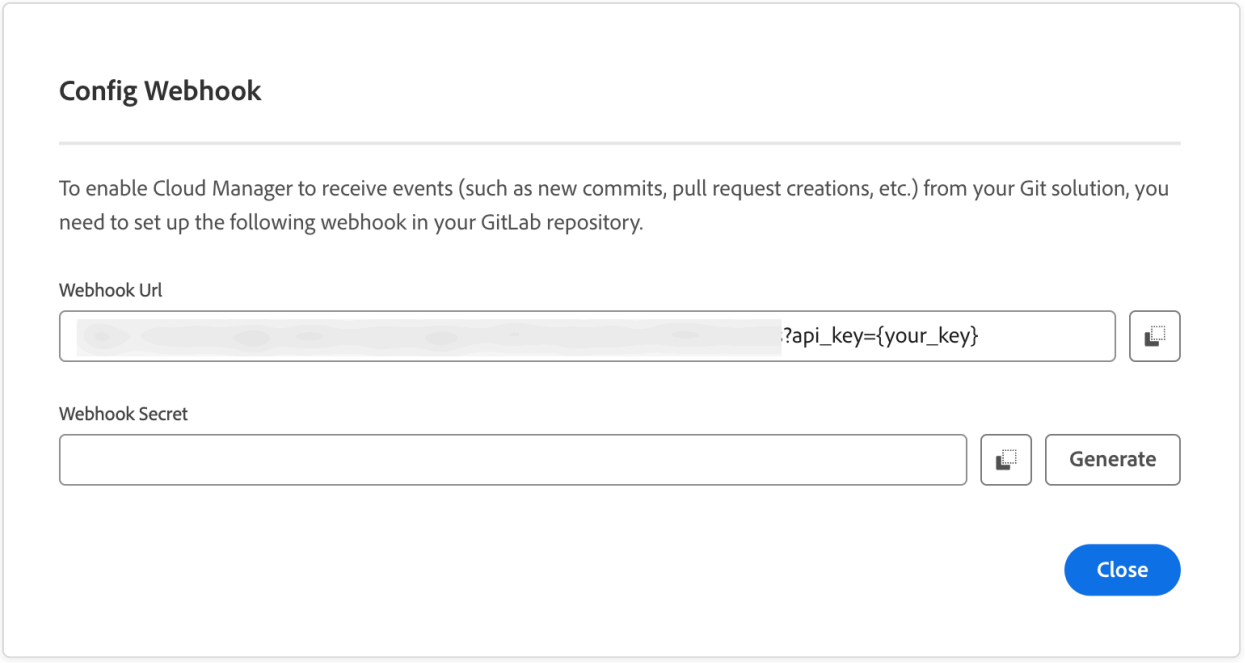
Source: Adobe Experience League — Add external repositories in Cloud Manager (token names adapted to the GE GitHub Enterprise repository)

3. Back on the **Repositories** page, open the **…** menu of the repository and click **Config Webhook**.



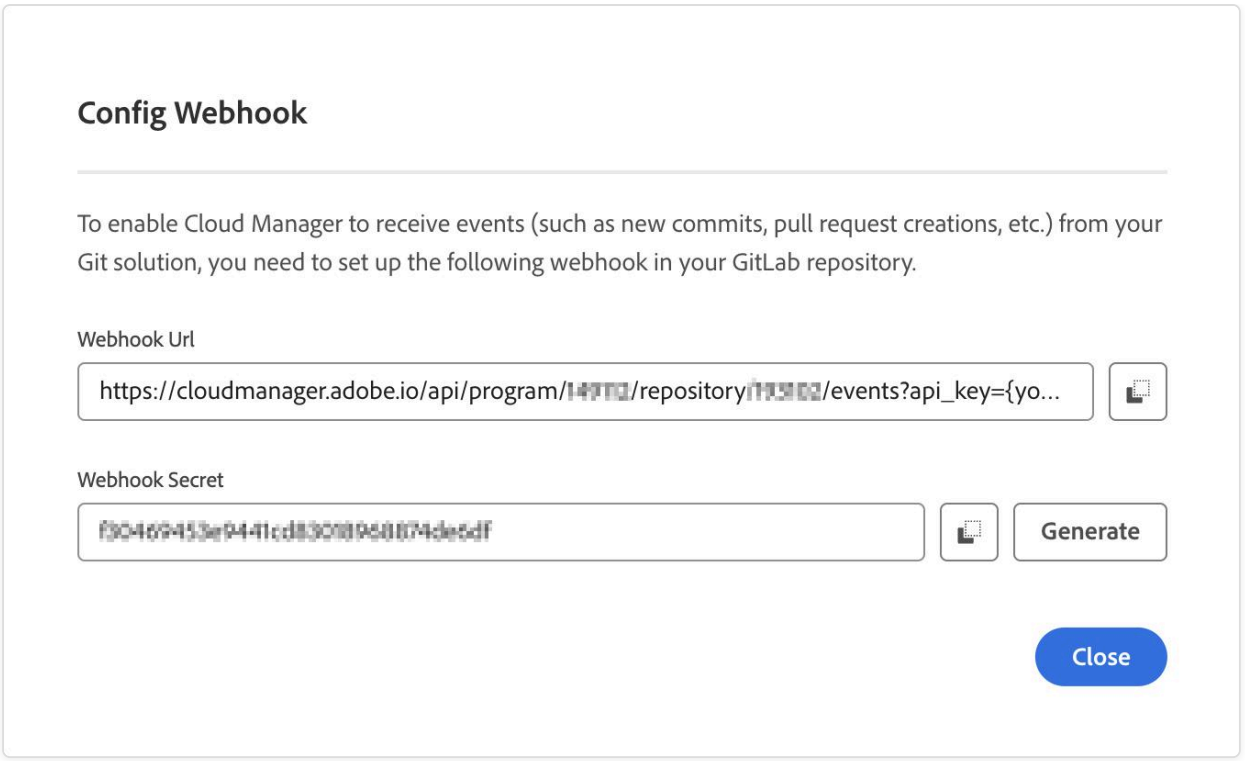
Source: Adobe Experience League — Add external repositories in Cloud Manager

4. In the **Config Webhook** dialog copy the **Webhook URL**; next to **Webhook Secret token/key** click **Generate** and copy the secret. Click **Close**.



Source: Adobe Experience League — Add external repositories in Cloud Manager

5. On the GitHub Enterprise server open the repository → **Settings** → **Webhooks** → **Add webhook**:
   - **Payload URL** — the Webhook URL from Cloud Manager, with its `api_key` query parameter replaced by your Adobe Developer Console API key;
   - **Content type** — `application/json`;
   - **Secret** — the Webhook Secret from Cloud Manager;
   - **Events** — select **Pull requests**, **Pushes** and **Issue comments**;
   - save the webhook.



Source: aem.live Bring Your Own Git

The webhook is what tells Cloud Manager about new commits, so Edge Delivery code sync depends on it.

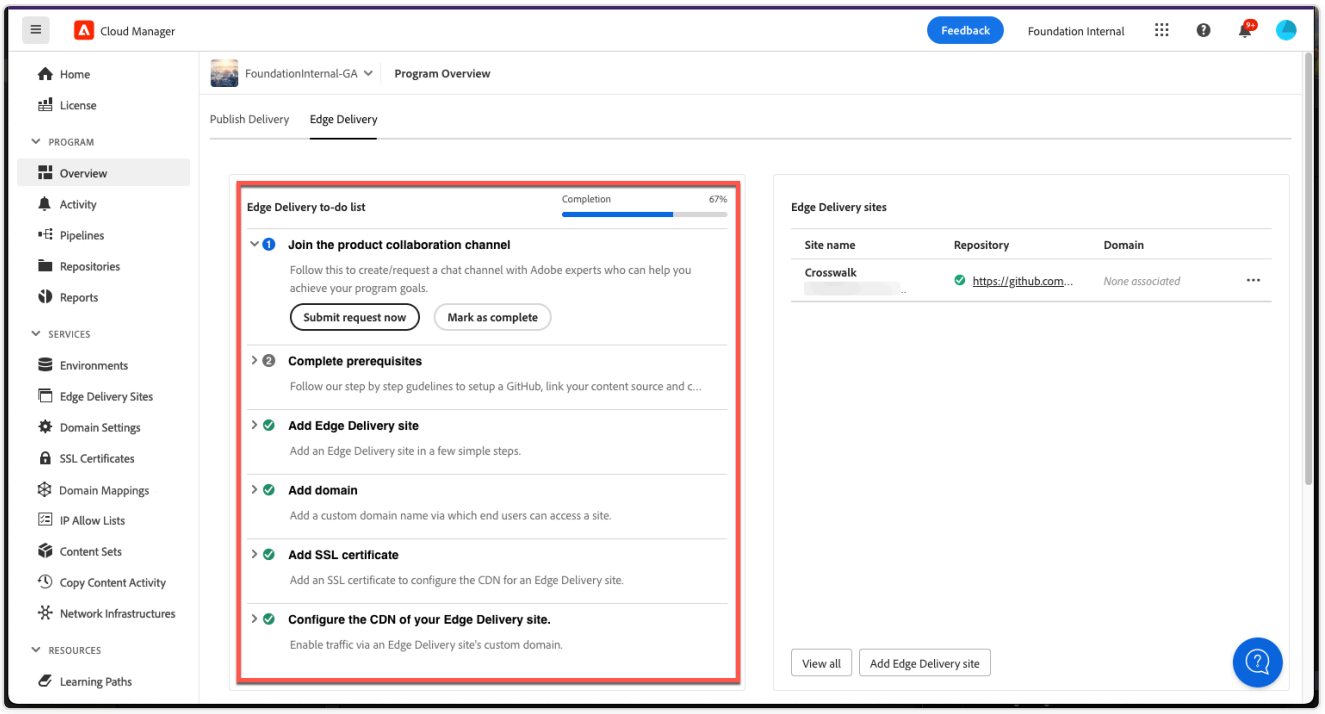
6. Wait until the repository is **Ready** on the Repositories page — only Ready repositories can be selected for Bring your own Git (B6).

B4. Find the program id and repository id

- **Program id**: visible in the Cloud Manager URL, e.g. `…/program/<program-id>/…`.
- **Repository id**: open **Repositories**, open the browser developer tools → **Network**, reload, and inspect the repositories API (XHR/fetch) response — each repository entry carries its numeric `id` (here `<repository-id>`). The id also appears in the URL when you open a repository's details.

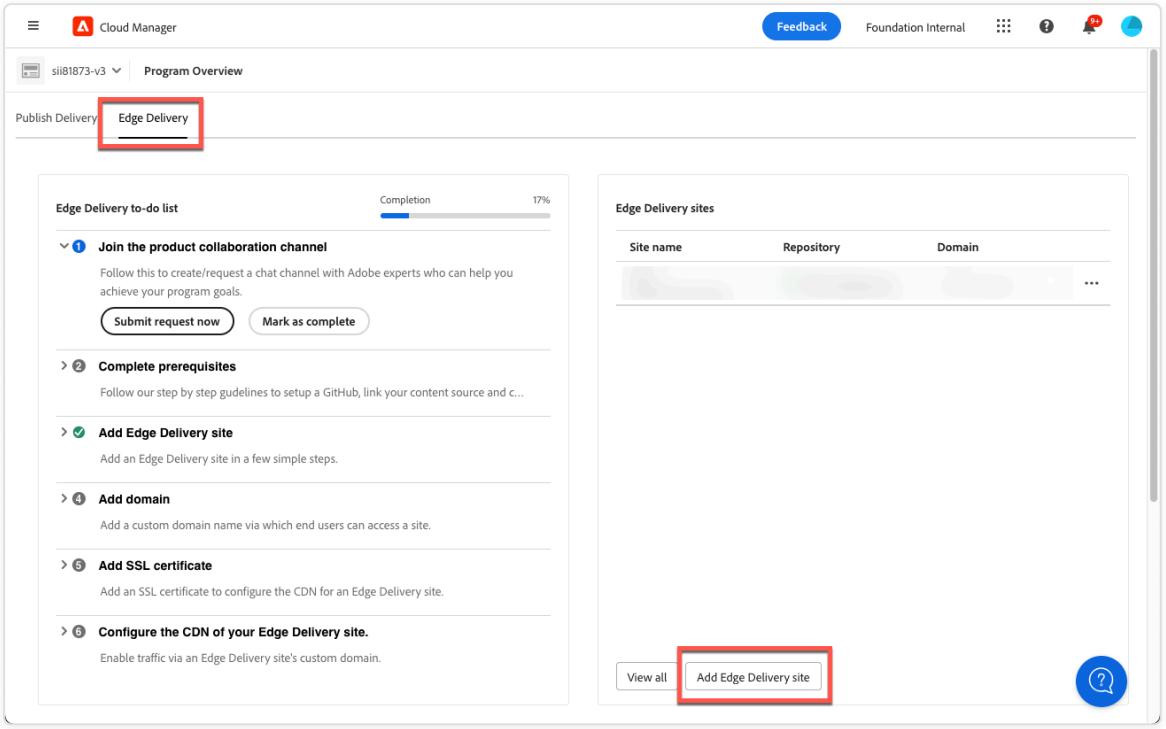
B5. Add the Edge Delivery site in Cloud Manager

The program must have an **Edge Delivery Services license**. The **Edge Delivery** tab shows a to-do list that walks through adding a site, a domain and the CDN:



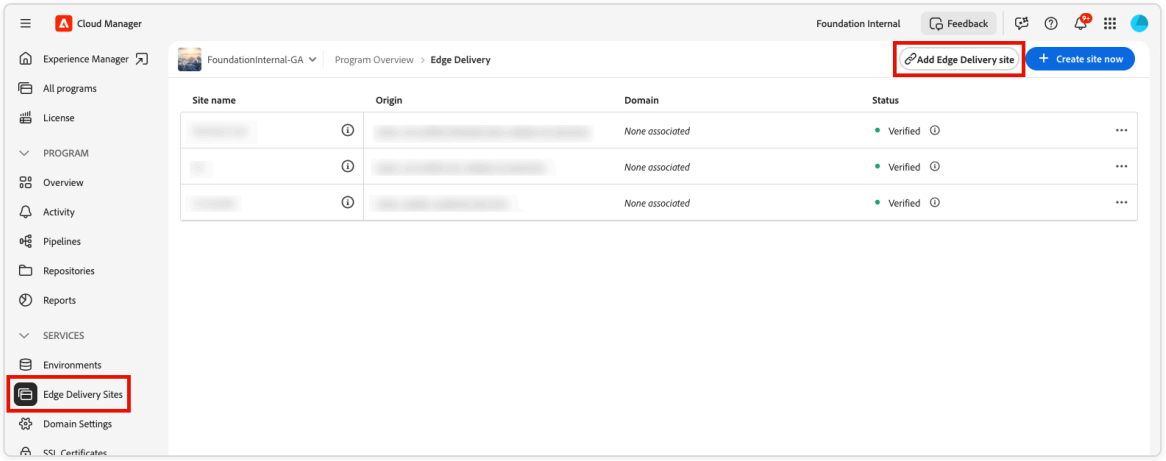
Source: Adobe Experience League — Introduction to Edge Delivery Services in Cloud Manager

1. Start **Add Edge Delivery site** in one of two ways:
    - **Program Overview** → **Edge Delivery** tab → **Add Edge Delivery site** (lower-right):



Source: Adobe Experience League — Add an Edge Delivery site to Cloud Manager

- or the side menu → **Services** → **Edge Delivery Sites** → **Add Edge Delivery site** (upper-right):

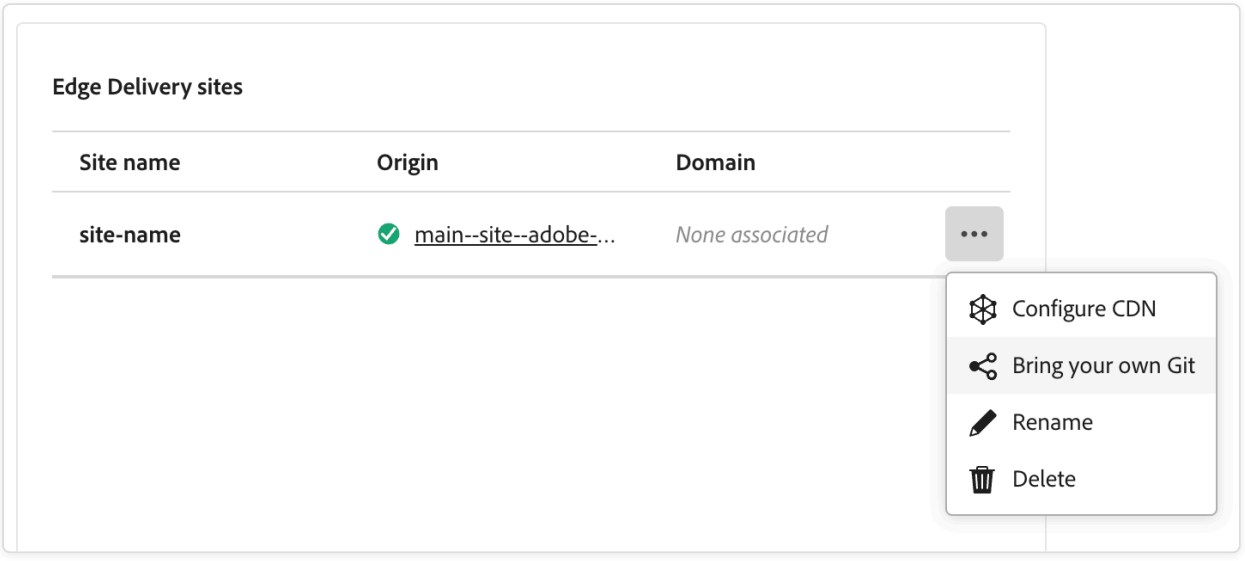


Source: Adobe Experience League — Add an Edge Delivery site to Cloud Manager

2. In the **Add Edge Delivery site** dialog enter:

   - **Site Name** — `geh-brand` (unique identifier in Cloud Manager);
   - **Edge Delivery origin** — `https://main--geh-brand--gehealthcare-eds.aem.live`;
   - **Site description** (optional).

   Click **Add**.



Source: aem.live Bring Your Own Git

3. In the **Verify repository ownership** dialog:

   1. add the file `well-known/adobe/cloudmanager-challenge.txt` (no leading period) to the `main` branch of the repository shown in **Repository URL**;

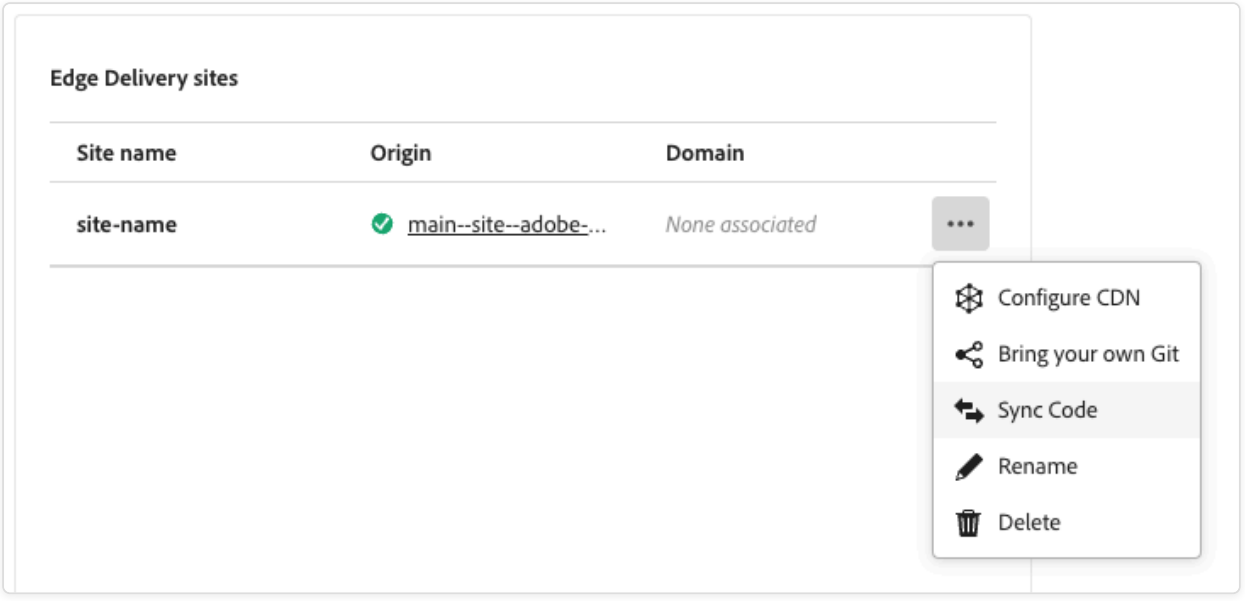
2. paste the code shown in step 2 of the dialog into that file;
3. commit it to `main` (via a pull request if the branch is protected);
4. click **Verify**.

> If the site uses site authentication, the challenge file cannot be read. Turn authentication off for the verification and back on afterwards.

> If the site is created with Cloud Manager's one-click flow (**Create site now**), you do not get admin rights in the site configuration and some Admin API calls will be refused. Creating the site first through the GitHub/Code Sync route (Part A) avoids that.

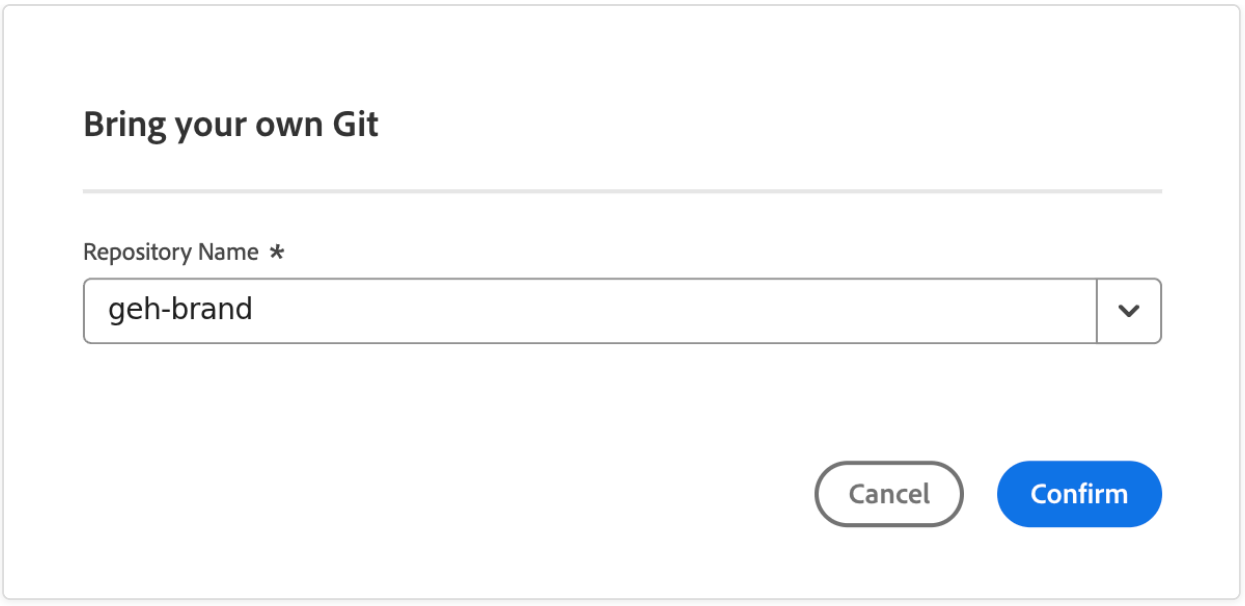
B6. Select "Bring your own Git" — use the repository for the site

1. **Program Overview** → **Edge Delivery** tab. In the **Edge Delivery sites** table click **…** at the end of the `geh-brand` row and choose **Bring your own Git**.



Source: aem.live Bring Your Own Git

2. In the **Bring your own Git** dialog choose the GitHub Enterprise repository `geh-brand` in **Repository Name** (only repositories with **READY** status are listed) and click **Confirm**.



Source: aem.live Bring Your Own Git (repository name adapted to the GE GitHub Enterprise repository)

3. Cloud Manager shows a **one-time secret token**. Copy it to a password manager immediately — it cannot be shown again. If you configure Bring your own Git again for the site, Cloud Manager generates a new token, and you must update the `cm-byog` secret (B7).

B7. Point the site's code source at Cloud Manager (Admin API)

```
export AEM_TOKEN='<admin-api-token>'      # see Part D — never commit or share

# 1) code source = repository onboarded in Cloud Manager
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/code.json" \
  -H "content-type: application/json" \
  -H "x-auth-token: $AEM_TOKEN" \
  --data '{
    "source": {
      "type": "byogit",
      "url": "https://cm-repo.adobe.io/api",
      "raw_url": "https://cm-repo.adobe.io/api/raw",
      "owner": "<program-id>",
      "repo": "<repository-id>",
      "secretId": "cm-byog"
    }
  }'

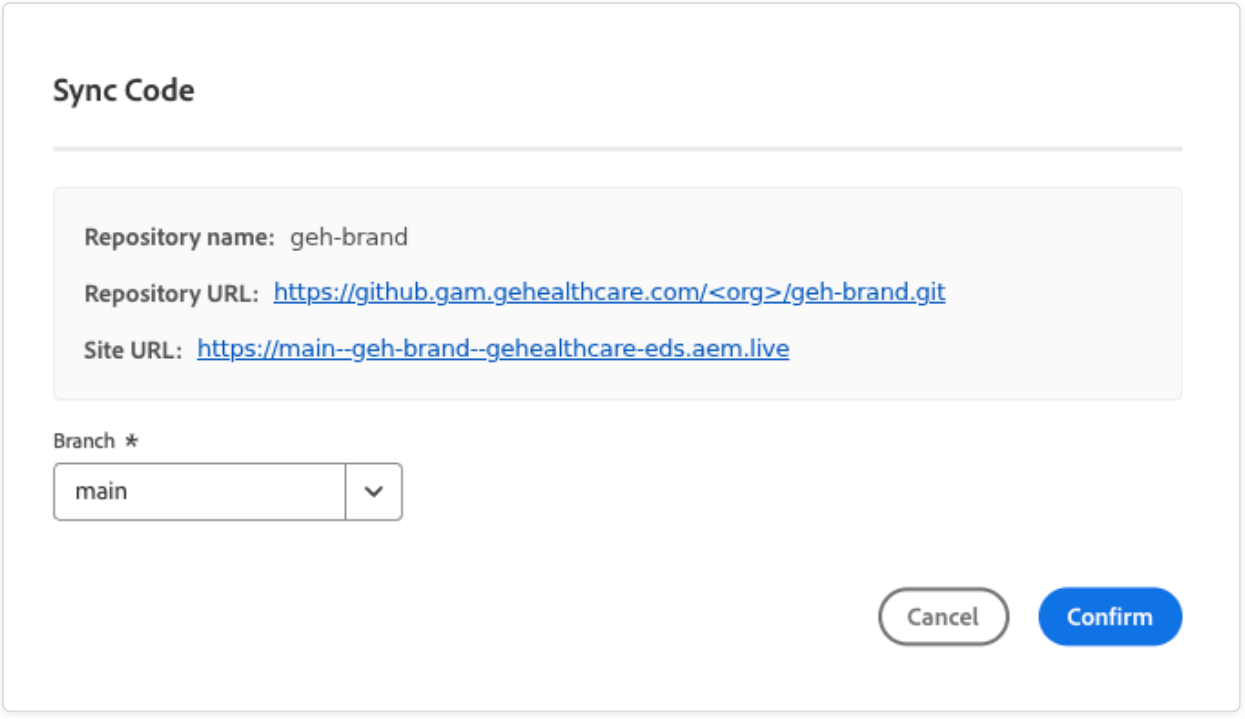
# 2) store the secret Cloud Manager gave you as "cm-byog"
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/secrets/cm-byog.json" \
  -H "content-type: application/json" \
```

```
  -H "x-auth-token: $AEM_TOKEN" \
  --data '{ "value": "<secret from cloud manager>" }'
```

`owner` = Cloud Manager **program id**, `repo` = **repository id** — Edge Delivery reaches the GitHub Enterprise repository through Cloud Manager (`cm-repo.adobe.io`), not directly.

B8. Sync Code — choose the branch

1. In the **Edge Delivery sites** table click **…** at the end of the site row → **Sync Code** (same menu as in B6).
2. The **Sync Code** dialog shows the **Repository name**, the **Repository URL** (the `github.gam.gehealthcare.com` URL) and the **Site URL**, and asks for a **Branch**:
   - the **Branch** list contains the branches that exist in the GitHub Enterprise repository;
   - choose **`main`** for the first sync — Edge Delivery needs a `main` branch, which serves `https://main--geh-brand--gehealthcare-eds.aem.page/`;
   - choose another branch (e.g. `feature-x`) to sync only that branch; it is then served at `https://feature-x--geh-brand--gehealthcare-eds.aem.page/`.



Source: aem.live Bring Your Own Git (repository and site values adapted to the GE GitHub Enterprise repository)

3. Click **Confirm**. After the first sync, each commit on **any** branch triggers an automatic sync through the webhook. Run **Sync Code** again only when you need a full manual sync (for example, after a sync error or after changing the secret).

   > If a branch is missing from the list, it does not exist in the GitHub Enterprise repository yet: push it (`git push ghe <branch>`) and open the dialog again. If commits stop syncing automatically, check the webhook (B3) — URL, API key, secret and events.

4. Check the sync job:

   ```
   curl -X GET "https://admin.hlx.page/job/gehealthcare-eds/geh-brand/main/code" \
     -H "content-type: application/json" -H "x-auth-token: $AEM_TOKEN"
   ```

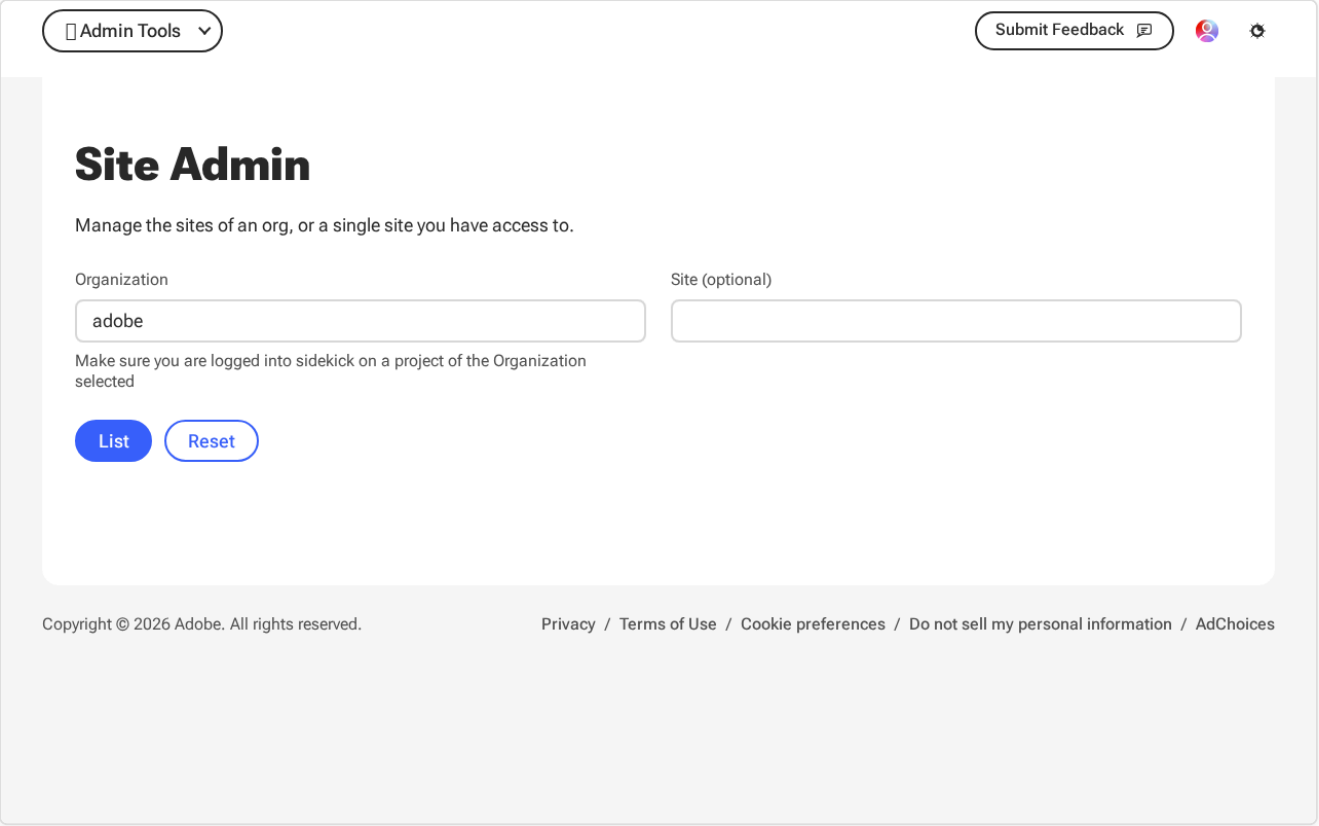
B9. Verify

- Open https://main--geh-brand--gehealthcare-eds.aem.page/ (use an incognito window — sync is asynchronous).
- Push a small visible change to `main` in the GitHub Enterprise repository and confirm it appears.
- **Branches:** a new branch is only served (`https://<branch>--geh-brand--gehealthcare-eds.aem.page/`) after it contains at least one commit pushed to the repository, so push a commit after creating it.
- After BYOG is active you can remove AEM Code Sync from the github.com repository from Part A; the code is now delivered from the GitHub Enterprise repository through Cloud Manager.

Part C — Check and maintain everything in tools.aem.live

Tool	Use it for
Site Admin — https://tools.aem.live/tools/site-admin/index.html	View/edit each site's configuration: code source (BYOG), **content source URL**,

Tool	Use it for
	CDN, access. Change a site's content source here instead of `fstab.yaml`.
User Admin — https://tools.aem.live/tools/user-admin/index.html	Admins, authors, publishers, and technical accounts (Config Admin) per site.



Read a site configuration from the command line:

```
curl -s "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-qa.json" -H
"x-auth-token: $AEM_TOKEN"
curl -s "https://admin.hlx.page/config/gehealthcare-eds/sites.json"           -H "x-
auth-token: $AEM_TOKEN"   # list sites
```

Part D — Admin API access token

- Admin API calls need `x-auth-token: <token>` (or `Authorization: token <token>`), issued to a user/key with the **admin** role on the org or site.
- Long-lived **API keys** are created per site (requires an admin-authenticated request):

```
curl -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/apiKeys.json" \
  -H "content-type: application/json" -H "x-auth-token: $AEM_TOKEN" \
  --data '{ "description": "BYOG setup", "roles": ["admin"] }'
```

The key value is returned **once** — store it securely; keys expire and should be rotated (see https://www.aem.live/docs/admin-apikeys).

Part E — One repository, many sites and content sources

Each environment (dev/QA/stage/prod) is a **repoless site** in the same org that reuses the onboarded GitHub Enterprise repository and points to its own AEM author environment.

E1. Prepare the AEM side of the new environment

1. In the target AEM author environment (e.g. `author-p<program-id>-e<qa-env-id>` for QA) create the site from the template (Part A4) with site name `geh-brand-qa` and Project URL `https://main--geh-brand-qa--gehealthcare-eds.aem.page`.
2. Copy that environment's **technical account ID** (Part A5).

E2. Create the site configuration (code + content + access)

`PUT` creates a new site; `POST` to the same URL updates an existing one.

```
curl -v -X PUT "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-
qa.json" \
  -H "x-auth-token: $AEM_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{
    "code": {
      "source": {
        "type": "byogit",
        "owner": "<program-id>",
        "repo": "<repository-id>",
        "url": "https://cm-repo.adobe.io/api",
        "raw_url": "https://cm-repo.adobe.io/api/raw",
        "secretId": "cm-byog"
```

```
      }
    },
    "content": {
      "source": {
        "url": "https://author-p<program-id>-e<qa-env-
id>.adobeaemcloud.com/bin/franklin.delivery/gehealthcare-eds/geh-brand-qa/main",
        "type": "markup",
        "suffix": ".html"
      }
    },
    "access": {
      "admin": {
        "role": {
          "admin": ["<admin-email>"],
          "config_admin": ["<technical-account-id>@techacct.adobe.com"]
        },
        "requireAuth": "auto"
      }
    }
  }'
```

What to change per environment:

Field	Value
URL path `sites/<site>.json`	the new site name, e.g. `geh-brand-qa`, `geh-brand-stage`, `geh-brand-dev`
`code.source.owner` / `repo`	Cloud Manager program id / repository id — **the same for every site**
`content.source.url`	`https://<that env's author>/bin/franklin.delivery/gehealthcare-eds/<site>/main`
`access.admin.role.admin`	the people who administer the site config
`access.admin.role.config_admin`	the **technical account of that AEM environment** (`…@techacct.adobe.com`)

Notes:

- A Cloud Manager repository can be *linked* (Bring Your Own Git in Cloud Manager) to **one** site only; linking it to another site revokes the first site's secret. Additional sites must use this repoless configuration.
- The official repoless example for extra sites omits `secretId`; if the extra site cannot fetch code, add the `cm-byog` secret to that site as in B7.

- The site is available immediately at `https://main--geh-brand-qa--gehealthcare-eds.aem.page/` (only a 404 until content is previewed). Local development: `aem up --url https://main--geh-brand-qa--gehealthcare-eds.aem.page`.

E3. Path mappings (only if required)

Path mappings translate AEM content paths to site URLs. Set them in the site's **public** configuration:

```
# add mappings
curl -i -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-stage/public.json" \
  -H "x-auth-token: $AEM_TOKEN" -H "Content-Type: application/json" \
  --data '{"paths":{"mappings":["/content/geh-brand/:/","/content/geh-brand/metadata:/metadata.json"],"includes":["/content/geh-brand/","/content/dam/geh-brand/"]}}'

# extended variant (site configuration + DAM mapped to the root)
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-stage/public.json" \
  -H "x-auth-token: $AEM_TOKEN" -H "Content-Type: application/json" \
  --data '{"paths":{"mappings":["/content/geh-brand/:/","/content/geh-brand/configuration:/.helix/config.json","/content/geh-brand/metadata:/metadata.json","/content/dam/geh-brand/:/"],"includes":["/content/geh-brand/","/content/dam/geh-brand/"]}}'

# remove the public configuration (and with it the path mappings)
curl -X DELETE "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-qa/public.json" \
  -H "x-auth-token: $AEM_TOKEN"
```

- `public.json` is readable by anyone at `https://<host>/config.json` — **never put secrets in it**.
- `mappings` entries are `<AEM path>:<site path>`; `includes` lists the AEM trees that may be previewed/published.

E4. Verify each environment

```
# code served from the shared repository
curl -sI "https://main--geh-brand-qa--gehealthcare-eds.aem.page/scripts/aem.js"

# a published DAM asset (example)
```

```
curl -sI "https://stage--geh-brand-stage--gehealthcare-eds.aem.page/content/dam/geh-brand/videos/media_1dbf487cb180019132eb137a1c13e9c03195e36ce.mp4"
```

A `200` on the asset URL confirms the asset was published and is served through the stage site.

Troubleshooting

Symptom	Check
Site still shows old code after a push	Webhook delivery in Cloud Manager (external repos); then the code job: `curl https://admin.hlx.page/job/<org>/<site>/<branch>/code -H "x-auth-token: $AEM_TOKEN"`
Job error `Unable to fetch hlxignore: 401`	The `cm-byog` secret is wrong — re-create it (B7) with the value from Cloud Manager
Admin API returns 401/403	Token expired or user lacks the admin role on the site — check in User Admin
New branch URL returns 404	Push at least one commit to the branch
Second site lost code access after linking in Cloud Manager	A repository can be linked to one site only; use the repoless configuration (Part E) for the others
Publishing from AEM fails	The environment's technical account is missing as Config Admin (A5 / `config_admin` in E2)

Checklist per new environment

1. Create the AEM site from the template in that environment (E1).
2. Copy the environment's technical account ID.
3. `PUT` the site configuration with the shared program/repo ids, that environment's content URL, admins and technical account (E2).
4. Add path mappings if the site needs them (E3).
5. Preview a page and verify code and assets (E4).